

Patch Now, Pay Later? Cybersecurity Vulnerabilities and Technical Debt ^{*}

João Dimas[†] Lampros Pnevmatikos[‡]

October 6, 2026

Abstract

We study how cybersecurity vulnerabilities affect firm performance. We analyze 551 open-core software firms from 2010 to 2025, comparing firms affected by vulnerability discoveries with unaffected firms. These discoveries concern third-party dependency packages, making their timing plausibly exogenous to the firms using them. Exposed firms experience approximately 10% slower headcount growth relative to their pre-treatment average and a 1.7 percentage point decline in acquisition probability. Developer activity shifts toward remediation: the share of non-remediation work falls by 2.8 percentage points, while elevated fix-related activity persists for twelve months. These changes accompany faster accumulation of technical debt and cognitive complexity, which grow approximately 20 and 16–21 percentage points faster, respectively. Together, the findings suggest that cybersecurity vulnerabilities can disrupt firm growth and place persistent demands on engineering resources.

^{*}We are grateful to Todd Gormley and Margarita Tsoutsoura for their invaluable comments and guidance throughout this project. We also thank Manthos Delis, Julie Zhiyu Fu, Armando Gomes, Spyros Lagaras, Asaf Manela, Emmanuel Tsiritakis, and participants of the Analysis Sync Seminar at Roblox for their insightful feedback and suggestions. We gratefully acknowledge support from the Google Research Program.

[†]Olin Business School at Washington University in St Louis.

[‡]Olin Business School at Washington University in St Louis. Send correspondence to: l.pnevmatikos@wustl.edu.

1 Introduction

“There are two types of companies: those that have been hacked, and those who don’t know they have been hacked.” — John T. Chambers, former CEO of Cisco

Software firms build their products using third-party packages that supply common functionality. A newly discovered vulnerability in one of these packages can require firms to redirect engineering resources toward urgent repairs, even when no attack has occurred. The immediate task is to remove the security risk, but updating a dependency may also require changes elsewhere in the product. These demands raise an economic question: how do cybersecurity vulnerabilities affect firm growth, and can the engineering response create costs that persist after the initial vulnerability is addressed?

Existing research establishes that cybersecurity risk matters for firms and investors. [Florackis et al. \(2023\)](#) show that risk measures constructed from corporate disclosures predict future cyberattacks and explain cross-sectional return variation. Such measures depend on what managers report, and firms may withhold information about severe incidents ([Amir et al., 2018](#)). [Crosignani et al. \(2023\)](#) trace the economic consequences of a major cyberattack through supplier networks. These studies illuminate the pricing of cyber risk and the consequences of realized attacks. Vulnerability discoveries raise a related question: whether responding to a security flaw imposes economic costs even before it is exploited. Studying this margin requires observing both the vulnerable software and the firm’s response.

A potential source of persistent costs is technical debt (TD). Technical debt refers to the future cost of reworking code that was originally written quickly to meet short-term goals ([Cunningham, 1992](#)). Similar to financial debt, firms effectively “pay interest” on technical debt through elevated maintenance costs ([Sculley et al., 2014](#)). They may eventually need to refactor the code, restructuring it without changing its functionality. Technical debt is widely recognized in software engineering ([Kruchten et al., 2012](#); [Li et al., 2015](#)), but it does not appear on financial statements. Measuring this liability requires access to the codebase and a consistent way to estimate the work needed to remedy coding problems. This makes software maintenance an important, but difficult to observe, margin of firm performance.

We study these questions in open-core software firms, which commercialize products whose core source code is publicly available. We identify 648 firms and obtain codebase data for 551, constructing a panel covering 2010 to 2025. The setting allows us to observe software development histories alongside firm outcomes. We combine dependency information and vulnerability records with employment histories from LinkedIn and financing and acquisition records from PitchBook. We also analyze 84,831 monthly repository snapshots using SonarQube, an industry-standard static code analysis tool. These snapshots let us measure technical debt and cognitive complexity, which captures how difficult the code’s logic is to understand and modify.

Our source of variation is the public discovery of critical vulnerabilities in third-party dependency

packages. These discoveries concern shared software components used by many firms. Their timing is therefore plausibly exogenous to the quality or performance of any one firm using the package. We identify exposure by matching firms' dependency versions to the versions affected by each vulnerability. We focus on critical-severity discoveries, rated at least nine on the Common Vulnerability Scoring System's ten-point scale. This yields 88 vulnerability events affecting 121 firms, approximately one in five firms in the broader sample.

Remediating a vulnerable dependency can involve more than installing an update. Developers may need to modify integration code, resolve incompatibilities with other components, and test whether the product still functions as intended. Time pressure can encourage expedient fixes that increase future maintenance needs. Yet remediation may also prompt teams to improve existing code, and additional development activity can help absorb the immediate workload. The effect on the codebase and firm performance is therefore an empirical question. Technical debt and developers' recorded work allow us to examine how firms respond and whether the response extends beyond the initial repair.

We estimate a stacked difference-in-differences model comparing changes in firms exposed to a critical vulnerability discovery with contemporaneous changes in unaffected firms. Firm-cohort fixed effects account for persistent differences across firms within each comparison, while cohort-specific time effects absorb common changes around each discovery. The identifying assumption is that exposed and unaffected firms would have followed parallel outcome trends in the absence of the discovery. This comparison separates the timing of an external vulnerability discovery from the concern that vulnerabilities in a firm's own code reflect its engineering quality.

Exposed firms experience approximately eight percentage points lower headcount growth than unaffected firms. This difference is equivalent to about 10% of treated firms' average pre-treatment growth rate. Acquisition probability declines by 1.7 percentage points. We find no statistically significant effects on funding or survival. The findings indicate that the economic consequences of vulnerability discoveries extend beyond the immediate security task, with slower organizational growth and fewer acquisition opportunities over the period we study.

We then examine whether these changes accompany a growing maintenance burden. Technical debt accumulates approximately 20 percentage points faster in exposed firms in the baseline specification. Cognitive complexity grows 16–21 percentage points faster across specifications. Technical debt summarizes the estimated future effort needed to remedy coding problems, while cognitive complexity captures the difficulty of understanding control flow and nested logic. Together, these measures show that the affected codebases accumulate larger maintenance liabilities and become harder to work with. This pattern is consistent with time-sensitive remediation creating persistent engineering costs, although the aggregate measures do not identify which individual patches produced the additional debt or complexity.

The development histories reveal both an immediate response and a longer shift in the composition of work. Individual code changes, or "commits," increase by approximately 2.7 per day

in treated repositories relative to controls immediately following discovery. The increase attenuates in wider event windows. Commit messages containing vulnerability-resolution language, such as “upgrade,” “CVE,” and “security-patch,” also become more common during the first two weeks. The concentration of this activity around discovery is consistent with an urgent remediation response.

The response continues beyond that initial burst. The share of commits containing fix-related language rises by 3.2 percentage points in the broader repository sample and remains elevated over the following twelve months. The share of non-remediation work falls by 2.8 percentage points. Feature-related commit shares show suggestive declines at longer horizons, while documentation-related shares show no detectable change. These patterns suggest that remediation places persistent demands on engineering resources. Commit descriptions reveal the composition of recorded activity; they do not directly measure developer time or the amount of new functionality delivered. The evidence is therefore suggestive of resource diversion, rather than a direct estimate of forgone innovation.

We also examine how code complexity relates to subsequent firm performance in the broader panel. A 10% increase in cognitive complexity is associated with a 1.18–1.32 percentage point reduction in two-year-ahead headcount growth. This relationship persists after accounting for firm, time, and main programming language effects, as well as new high-severity security issues and security hotspots. This comparison is correlational. Read alongside the vulnerability results, it is consistent with a role for maintenance frictions in future growth, but it does not establish that complexity mediates the effect of vulnerability discoveries.

We assess the sensitivity of the main comparisons using permutation tests, alternative clustering structures, different event windows, and leave-one-cohort-out analyses. We also include industry-segment by calendar-month fixed effects to account for changes specific to firms’ product markets. The results section presents these checks alongside event-study evidence so that the timing and stability of the estimated differences can be assessed (Section 3.4).

Our results contribute to several strands of the literature. First, we extend research on the economic consequences of cybersecurity risk from realized attacks and disclosure-based measures to vulnerability discoveries in shared software dependencies (Florackis et al., 2023; Amir et al., 2018; Crosignani et al., 2023). Direct codebase observation allows us to identify affected firms and study their engineering response. This setting brings the operational costs of responding to vulnerabilities into the analysis of cybersecurity and firm performance.

Second, we contribute to research on intangible assets and the liabilities associated with software. Prior work documents the importance of intangible capital for firm value and growth (Ewens et al., 2025) and the pricing of organization capital (Eisfeldt and Papanikolaou, 2013). Software assets also require future maintenance, a liability that is difficult to observe in conventional financial data. We quantify technical debt using static code analysis and express the estimated remediation burden in dollar terms. Linking these measures to vulnerability discoveries shows how an external security shock can accompany changes in the future resources needed to maintain a firm’s software, with

applications to valuation and M&A due diligence.

Third, we provide evidence on how firms’ engineering responses to adverse events can persist. The immediate increase in commits is followed by a sustained shift toward fix-related activity and faster accumulation of technical debt and complexity. Observing these margins together connects a security event to the continuing work of maintaining a product. The findings suggest that cybersecurity vulnerabilities can constrain growth through persistent demands on engineering resources, while leaving the precise contribution of each engineering channel for further investigation.

The remainder of the paper proceeds as follows. Section 2 describes our data sources, sample construction, and measurement approach for both cybersecurity shocks and technical debt. Section 3 presents our empirical framework and main results, including the effects of critical vulnerability discoveries on firm outcomes, an analysis of technical debt and cognitive complexity as mechanisms, commit-level evidence of remediation behavior, and robustness tests. Section 4 concludes.

2 Data & Background

This section describes our data and the key approaches used in constructing our shocks and metrics. We begin by introducing our sample of technology firms, explaining what the open-core business model is and why it is crucial for our analysis. Afterward, we describe the firms’ characteristics, including their geographic distribution and industry composition. We then detail our four primary data sources: firm codebases from GitHub and GitLab, employee headcount from LinkedIn, vulnerability data from the GitHub Advisory Database and National Vulnerability Database, and funding and acquisition information from PitchBook. Next, we explain our methodology for identifying cybersecurity shocks and matching them to affected firms using automated software composition analysis. Finally, we introduce technical debt as a key measure of hidden software liabilities and describe how we quantify it using industry-standard static code analysis.

The primary challenge in studying firms’ technological assets is that codebases are typically proprietary and accessible only to internal stakeholders. To overcome this, we focus on firms that utilize an open-core business model—companies that make their core product code publicly available while monetizing through enterprise features and support services. This approach allows us to construct a novel panel linking complete codebase histories to firm-level outcomes over a 15-year period.

2.1 Sample Construction and Firm Characteristics

Open-source software (OSS) has become foundational infrastructure for the global economy. Industry audits reveal that 96% of commercial codebases contain open-source components, and that open source constitutes approximately 77% of all code in the average application ([Synopsis, 2024](#)). [Hoffmann](#)

et al. (2024) estimate the demand-side value of widely used OSS at \$8.8 trillion—implying that firms would need to spend 3.5 times more on software development if these components did not exist. This pervasiveness means that software produced by even small open-source firms can become deeply embedded in the technology stacks of large enterprises, financial institutions, and government agencies. A vulnerability in a widely adopted open-source database engine or authentication library does not merely affect its developer; it propagates downstream to every organization that depends on the product, creating systemic exposure across the software supply chain (Crosignani et al., 2023).

The firms in our sample are precisely the producers of such foundational software. They operate under an open-core business model, which combines the collaborative benefits of OSS with revenue opportunities from commercial licensing and support. Under this model, companies release their fully functional core product under an open-source license—making the code freely accessible to developers, customers, and researchers—while maintaining exclusive rights to license enterprise-grade features, modules, or dedicated support services through paid subscriptions. The organizational literature highlights OSS as a central driver of innovation in the modern economy (Conti et al., 2025, 2023; Fershtman and Gandal, 2011), and many companies transition from fully open-source projects to the open-core model as they seek to generate revenue while preserving community engagement and collaborative development (Nagle, 2019). A tangible example is *RStudio* (now *Posit*), the primary development environment for the *R* programming language used by researchers, data scientists, and analysts across industries. The software is freely available and open source, but the company monetizes enterprise deployment through commercial licenses and a cloud platform. Crucially, a security flaw in *RStudio*’s codebase would affect not only *Posit* as a firm but also the thousands of organizations whose analytical workflows depend on the product.

Although releasing product code publicly may appear to invite competition, this strategy offers significant advantages: public availability accelerates adoption, fosters external contributions from developers worldwide, and builds trust through transparency. When individuals learn to use freely available software, they often advocate for enterprise adoption at their employers, creating a pathway to commercial sales. Crucially for our research, this business model grants us access to detailed codebase histories that would otherwise remain confidential within traditional software companies.

We construct our sample by identifying a universe of companies that commercialize OSS repositories, sourced through OSSrank, a leading platform in the OSS ecosystem. This process yields 648 unique companies and the 734 distinct open-source repositories they develop and commercialize. As Table 1 below illustrates, these firms produce foundational technology products (databases, cloud infrastructure, security tools, developer platforms, and analytics systems) that serve as building blocks in the software supply chains of larger organizations. Our firms are primarily based in the United States, operating predominantly in the technology sector. They are typically small, private, and frequently venture capital-backed. The median firm has 19 employees and was founded in 2013, with founding years ranging from 2005 (10th percentile) to 2020 (90th percentile). Traditional financial disclosures

are not publicly available for these private firms.

While these firms are small relative to publicly traded technology companies, two features make them well suited for studying the economic consequences of cybersecurity vulnerabilities. First, their products are widely adopted downstream: open-core databases, monitoring tools, and security libraries are integrated into the technology stacks of Fortune 500 companies, financial institutions, and critical infrastructure operators. A vulnerability in one of these products creates exposure not only for the producing firm but for every downstream adopter—precisely the supply-chain propagation channel that makes cybersecurity economically consequential (Hoffmann et al., 2024). Second, small firms are disproportionately vulnerable to the operational disruption that vulnerability remediation entails, because they lack the redundant engineering capacity that larger organizations can deploy during crises. The mechanism we study—cybersecurity vulnerabilities diverting engineering resources—applies to any organization maintaining a software codebase, regardless of size. Moreover, these firms are representative of the broader technology ecosystem: the vast majority of software firms are small, private, and venture-backed, and they collectively account for a substantial share of software innovation and supply-chain activity (Nagle, 2019). We note that our largest firms (90th percentile headcount exceeds 140 employees) exhibit qualitatively similar patterns, though statistical power for this subsample is limited.

Table 1 presents the distribution of firms across product categories. We employ two large language models (Gemini and GPT) to independently classify each firm’s core offering into one of ten product categories, retaining the consensus classification (86% inter-model agreement).¹ The sample exhibits substantial diversity: approximately 24% of firms develop developer tools and platforms, 15% provide cloud and infrastructure services, 14% focus on databases and storage, and 14% specialize in analytics and observability. The remaining firms span security, enterprise applications, AI and machine learning, collaboration, and testing. This diversity ensures that our findings are not driven by idiosyncrasies of a single product category.

¹For the 14% of firms where the two models disagree, we use the Gemini classification. Results are not sensitive to this choice; using GPT classifications or restricting to agreement-only firms yields qualitatively identical category distributions.

Table 1: Firm Types (Based on Core Product)

Firm Type	Frequency (%)	Cumulative (%)
Developer Tools & Platforms	24.2	24.2
Cloud & Infrastructure	14.5	38.7
Databases & Storage	14.1	52.9
Analytics & Observability	13.5	66.4
Security & Identity	10.9	77.3
Enterprise & Business	5.5	82.8
AI & Machine Learning	5.5	88.3
Collaboration & Productivity	4.9	93.2
Other	4.5	97.7
Testing & QA	2.3	100.0

Table 2 presents the geographic distribution of our sample firms. Most of our firms are headquartered in the United States, accounting for 62% of the sample. Beyond the United States, our sample includes meaningful representation from major technology hubs in Europe and Asia. Germany (4%), the United Kingdom (4%), and France (3%) represent the largest European contingents, while Canada (2%) accounts for the primary North American presence outside the U.S. The remaining firms are distributed across technology centers including Israel, China, the Netherlands, and India. Countries with fewer than 0.5% of sample firms are grouped into the "Other" category. Geographic headquarters information is unavailable for 13% of sample firms.

Table 2: Geographic Distribution of Firms

Country	Frequency (%)	Cumulative Frequency (%)
United States	61.63	61.63
Germany	4.16	65.79
United Kingdom	3.54	69.34
France	3.08	72.42
Canada	2.16	74.58
Italy	1.85	76.43
China	1.39	77.81
Netherlands	1.23	79.04
Israel	1.08	80.12
Ireland	0.92	81.05
India	0.77	81.82
Other	4.93	86.75
Missing	13.25	100.00

2.2 Codebase Data

We match each project identified through OSSrank to its repository address using the GitHub and GitLab search APIs.

These repositories are hosted on GitHub or GitLab, the industry-standard platforms for software version control. These platforms maintain complete version histories through a system of *commits*. A commit is a discrete submission of code changes to the project—analogous to saving a new version of a document, but with comprehensive metadata attached. Each commit records exactly which lines of code were added, modified, or deleted; a timestamp; the identity of the contributor; and a *commit message*—a brief text description written by the developer explaining the purpose of the change (e.g., “Fix login bug” or “Add user authentication”). This granular audit trail allows us to observe not only the state of each codebase at any point in time, but also the intensity and content of development activity.

We build a monthly panel by extracting the final commit from each repository for every month from 2010 to 2025. Most repositories are hosted on GitHub, with a minority on GitLab; we combine both sources given the platforms’ functionally equivalent version control features. We successfully download and analyze 639 of the 734 repositories, corresponding to 551 distinct firms for which we observe commit-level development activity. For our code quality and technical debt analyses, we process each monthly snapshot through SonarQube static analysis (Section 2.7), successfully obtaining metrics for 488 of these firms; the remainder use programming languages or build configurations that

our static analysis pipeline does not support. Firm-level analyses requiring technical debt, cognitive complexity, and other code quality measures draw from this subset of 488 firms.

2.3 Employee and Funding Data

To track firm growth and outcomes, we construct monthly headcount series using employee-position data from LinkedIn. We obtain comprehensive employment histories for individuals listing positions at our 648 sample firms, including start dates, end dates, and standardized role titles. We match firms to their LinkedIn company pages and filter the universe of LinkedIn position records to those associated with our target companies. We aggregate to the firm-month level, counting unique employees present at each firm in each month. Additionally, we classify software engineering roles using a detailed mapping of job titles (e.g., "software engineer," "backend developer," "DevOps engineer") to construct a parallel measure of technical headcount. Our final employee panel spans 2010 to 2025 at monthly frequency.

We supplement employee data with funding and acquisition information from PitchBook, a leading private capital markets database. We link our sample firms with data on funding rounds (Seed Round, Early Stage VC (Series A, B, C), Late Stage VC (Series D+), PE Growth/Expansion, Accelerator/Incubator, Angel (individual)), initial public offerings (IPOs), mergers and acquisitions, and business status indicators (alive/not-alive). For each event, we observe the date and transaction type, enabling us to construct indicators for whether a firm has received funding, been acquired, or ceased operations. These outcome measures serve as proxies for firm performance in the absence of traditional financial statements, which are not publicly available for the private firms in our sample.

2.4 Vulnerability Data

For our causal analysis, we identify cybersecurity shocks that are exogenous to firm quality by focusing on vulnerabilities affecting third-party software packages that firms integrate into their code. These dependency vulnerabilities directly translate to security flaws in firms' core products, but crucially, their discovery is plausibly unrelated to any individual firm's engineering practices or management quality.

We draw vulnerability data from two complementary industry-standard sources. The GitHub Vulnerability Advisory Database, maintained by GitHub, serves as the primary vulnerability reporting mechanism for the open-source software ecosystem. When security researchers or maintainers discover vulnerabilities, they report them through this centralized system, creating a standardized, time-stamped record of public discovery. The National Vulnerability Database (NVD), operated by the U.S. National Institute of Standards and Technology (NIST), provides the official government repository for vulnerability information and supplies the authoritative scoring and classification used by enterprises and security vendors worldwide. While maintained by a U.S. government agency, the NVD catalogs vulnerabilities in software packages used globally, making it the international standard

for vulnerability tracking regardless of where affected firms are located. Both databases score vulnerabilities using the Common Vulnerability Scoring System (CVSS), an industry-standard framework that quantifies vulnerability severity on a 0–10 scale.

We source our data from the official GitHub Advisory Database repository, which incorporates merged vulnerability information from the NVD. To ensure accuracy and avoid false positives, we specifically utilize the "GitHub reviewed" subset of this database. Focusing solely on critical dependency vulnerabilities affecting the software our firms use, we identify 88 unique critical cybersecurity events that impacted 121 firms via their dependencies. Appendix Tables 19–21 summarize the broader GitHub-reviewed advisory universe from which we draw our shocks, including the distribution across severity levels, software ecosystems, and years.

The CVSS framework provides a standardized way to measure vulnerability severity based on characteristics such as how easily a vulnerability can be exploited (e.g., remotely over a network versus requiring physical access) and the potential impact on system confidentiality, integrity, and availability. Scores range from 0 to 10, with vulnerabilities scoring 9.0 or above classified as “critical.” Importantly, CVSS Base scores reflect the intrinsic severity of a vulnerability under standardized assumptions—they do not indicate whether exploitation occurred or whether any particular firm was compromised. By restricting our sample to critical-severity vulnerabilities, we study high-severity discovery events that are comparable in potential impact and likely to prompt organizational response. A detailed explanation of CVSS metrics, scoring methodology, and our operational procedures for handling different CVSS versions across data sources is provided in Appendix A.1.

2.5 Matching Shocks to Firms

To identify which firms are affected by specific vulnerabilities, we must first comprehensively map each firm’s software dependencies. Firms might not use the most recent version of a package for numerous reasons—for example, if essential functionality is not supported in the latest release. Accurate identification of exposure therefore requires knowing the exact dependency version a firm was using at the time of the shock. We use Syft, an industry-standard software composition analysis tool, to automatically scan and catalog all software components used by each firm at each point in time. Syft automatically scans codebases and generates complete “Software Bill of Materials” (SBOM) inventories—detailed catalogs of each software component and its version in a firm’s technology stack. Syft identifies the exact version of each dependency, which is crucial because vulnerabilities typically affect specific version ranges. For example, a vulnerability might affect versions 1.2.0 through 1.4.5 but not 1.5.0+.

We then match each critical-severity vulnerability shock to the firm that commercializes the affected repository.

2.6 Example of a Critical Dependency Shock

A security flaw was discovered in *xrpl.js*, a software component for cryptocurrency applications. This component was used by numerous companies across multiple software versions (2.14.2 through 4.2.4). The vulnerability allowed malicious actors to steal users’ private cryptographic keys, which is the cryptocurrency equivalent of bank account credentials. This creates a natural experiment for four key reasons. First, the vulnerability’s discovery was unpredictable, as it was found by an independent security researcher. Second, all companies using the affected software versions were simultaneously exposed (treated), regardless of their individual business strategies, management quality, or technical choices. Third, the response was mandatory, as no treated company could choose to ignore a vulnerability that allowed the theft of customer funds. Fourth, its magnitude was quantifiable using the CVSS standard, which scored this particular vulnerability 9.3/10, indicating critical severity.

2.7 Technical Debt: Background & Measurement

The development of new software products or features frequently generates technical debt (TD). The software engineering literature defines technical debt as coding decisions that accelerate short-run development at the expense of future changes and higher maintenance costs ([Sculley et al., 2014](#)). Like financial debt, firms “pay interest” through elevated future maintenance expenses and eventually face costly “principal paydowns” when refactoring becomes unavoidable. Recent estimates place the aggregate stock of U.S. technical debt at approximately \$1.5 trillion, underscoring its economic significance as an intangible liability ([Krasner, 2022](#)).

A central challenge for financial economists is that technical debt remains largely unobservable to external stakeholders. Two firms may deliver functionally identical products from a user’s perspective, yet differ dramatically in their accumulated software liabilities. Because direct measurement requires continuous access to proprietary source code, only internal developers and managers can accurately assess TD magnitude. This opacity makes technical debt difficult to contract upon, monitor, or price in financial markets, despite its economic importance. Survey evidence illustrates this burden: developers allocate approximately 13.5 hours per week to managing technical debt ([Stripe, 2018](#)).

Its effects, while indirect, can be economically significant: engineering resources are diverted from innovation to maintenance, and increasing code complexity slows feature development velocity. This friction is especially problematic for early-stage firms that must rapidly pivot business models or introduce new features to achieve product-market fit, often generating substantial TD in the process.

These costs persist well beyond the startup phase. Survey evidence from chief information officers at large enterprises (annual revenue exceeding \$1 billion) indicates that TD accounts for 20–40% of the total value of their technology assets ([McKinsey and Co, 2020](#)). Over two-thirds of these firms allocate more than 10% of new product development budgets to servicing existing debt rather

than building new capabilities. Benchmark analyses across 700 code repositories further illustrate the scale: TD remediation costs range from \$2.79 to \$11.77 per line of code, depending on programming language and architecture (Curtis et al., 2012). To contextualize these estimates, Google’s codebase comprises approximately 2 billion lines of code, implying potential TD liabilities in the tens of billions of dollars for large technology firms.

A critical feature of technical debt is its potential for nonlinear accumulation. As TD levels rise, the underlying software architecture can become increasingly complex, making subsequent changes more difficult and costly. This dynamic creates a feedback loop: high technical debt burden can erode a firm’s capacity to service existing software liabilities while simultaneously accelerating the rate at which new TD accumulates.

To measure TD, we employ SonarQube, an industry-standard static code analysis tool, using its SQALE (Software Quality Assessment based on Lifecycle Expectations) methodology (Letouzey, 2012; Dimas and Pnevmatikos, 2026). We scan each repository at monthly frequency, taking the final commit of each month as the state of the codebase for that period. SonarQube performs automated inspection of each snapshot, identifying maintainability issues (e.g., code duplication, excessive complexity, poor documentation) and assigning each issue a model-implied remediation time based on over 9,000 language-specific rules. For instance, a documentation violation might require 2 minutes to fix, while a structural problem could require several hours. Aggregating these remediation times across all maintainability issues yields the SQALE Index, measured in hours. Our analysis pipeline supports over 30 programming languages; Appendix A.3 provides implementation details for languages requiring specialized configurations.

While the SQALE Index provides a useful within-sample measure, its absolute level is not directly interpretable in economic terms: it depends on which rules are enabled and how the remediation model weights different issues. To convert the SQALE Index into dollar-denominated technical debt that is comparable across firms, we calibrate it to an external benchmark through a three-step procedure.

First, we construct a proxy for the replacement cost of each firm’s software assets. Let ℓ denote development time per non-comment line of code, which we set to $\ell = 0.5$ hours (30 minutes)—SonarQube’s default parameter. For a codebase with $\text{LoC}_{i,t}$ lines of code, the replacement cost in developer hours is Re-development Effort $_{i,t} = \ell \times \text{LoC}_{i,t}$. Multiplying by the hourly wage of a software engineer ($w = \$55$, based on Bureau of Labor Statistics data (U.S. Bureau of Labor Statistics, 2025)) yields Software Assets in dollars: $\text{Software Assets}_{i,t} = w \times \text{Re-development Effort}_{i,t}$.

Second, we choose a scaling factor κ that converts the raw SQALE Index into calibrated remediation effort. Survey evidence suggests that firms typically assess technical debt at 20–40% of their software assets (McKinsey and Co, 2020). We use the conservative lower end of this range and set κ so that the median firm’s technical debt equals 20% of its software assets. Formally, let $\varphi = 0.20$

denote this target ratio. We choose κ such that:

$$\text{Median}(w \times \kappa \times \text{SQALE}_{i,t}) = \varphi \times \text{Median}(w \times \ell \times \text{LoC}_{i,t}), \quad (1)$$

where medians are computed across all firm-month observations. Simplifying yields $\kappa = \varphi \times \text{Median}(\ell \times \text{LoC}_{i,t}) / \text{Median}(\text{SQALE}_{i,t})$. In our sample, this produces $\kappa = 33.22$, which we hold fixed throughout.

Third, we apply this calibration to construct Technical Debt for each firm-month:

$$\text{Technical Debt}_{i,t} = w \times \kappa \times \text{SQALE}_{i,t}.$$

Because κ is constant across firms and time, it rescales the SQALE Index proportionally, leaving statistical significance unchanged while making magnitudes economically interpretable. This approach yields a dollar-denominated measure comparable across firms and over time. Because it is computed from static analysis of source code, it can be constructed retrospectively for each monthly snapshot, allowing us to track the evolution of technical debt at high frequency over the 2010–2025 sample period.

Beyond technical debt, we employ *Cognitive Complexity* as a complementary measure of code-base integrity. While technical debt captures the estimated remediation cost of code issues, cognitive complexity quantifies the mental effort required to understand and modify code. This metric was developed by SonarSource and implemented in SonarQube to address limitations in earlier complexity measures such as Cyclomatic Complexity, which was designed to measure testability rather than understandability (McCabe, 1976; Campbell, 2023). Cognitive Complexity increments for structures that break linear code flow—loops, conditionals, and `catch` blocks—and applies additional penalties when such structures are nested, reflecting the increased cognitive load of understanding deeply nested logic. Unlike Cyclomatic Complexity, which assigns equal weight to all branching structures, Cognitive Complexity weights structures based on the mental effort they impose: a `switch` statement receives a single increment regardless of case count, while mixed boolean operators and recursion receive explicit penalties. This design yields scores that correspond more closely to programmers’ intuitions about code difficulty. In our regression analysis, we winsorize cognitive complexity at the 1st and 99th percentiles to limit the influence of outliers. Appendix A.5 provides a detailed explanation and discussion of this metric.

The left panel of Figure 1 presents a histogram of the natural logarithm of technical debt across firms’ codebases in the 1st quarter of 2024. This is calculated by summing Technical Debt across all of the repositories a firm commercializes. The median value of firm-level technical debt is approximately \$808,000.

The right panel of Figure 1 shows the size of the liability relative to the codebase, measured as

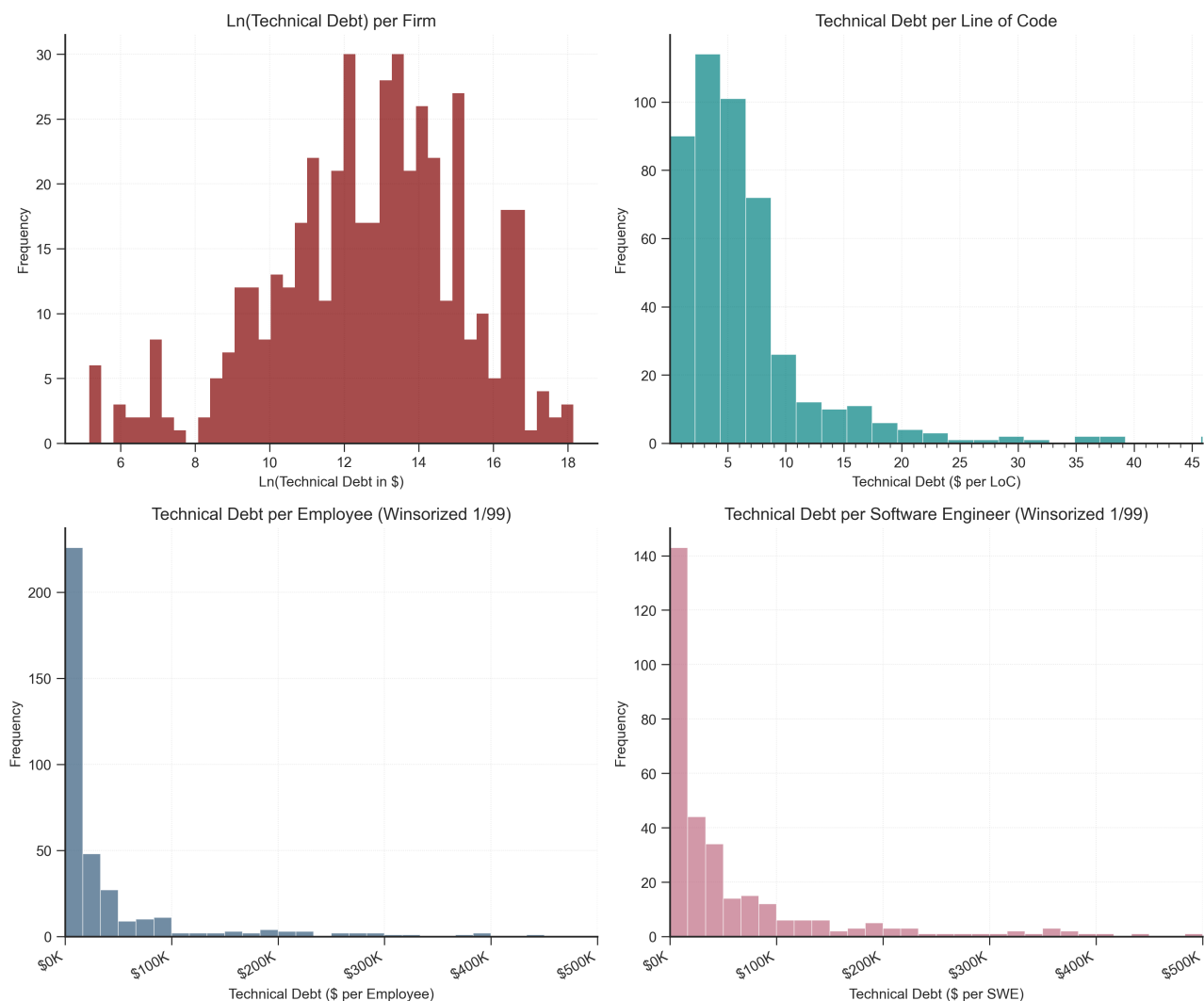


Figure 1: The Cross-Section of Firm-Level Technical Debt in Q1 2024

technical debt per line of code (LoC). This measure varies substantially, with an interquartile range of roughly \$4.07–\$11.85 per LoC. For context, Google’s codebase exceeds 2 billion lines. Therefore, the magnitude of this liability can be substantial. Table 3 reports summary statistics for the firm-level sample, including several of our novel code-based metrics.

2.8 Summary Statistics

Table 3: Summary Statistics - Firm-Month Panel

Variable	Distribution							
	N	Mean	SD	Min	P25	Median	P75	Max
Panel A: Outcome Variables								
Ln(Headcount)	27,827	3.696	2.644	0.000	1.869	3.219	5.034	12.669
Ln(SWE Headcount)	23,577	3.165	2.380	0.000	1.609	2.708	4.277	11.216
Funding Round (0/1)	53,593	0.012	0.111	0.000	0.000	0.000	0.000	1.000
Acquisition (0/1)	53,593	0.043	0.204	0.000	0.000	0.000	0.000	1.000
Is Alive (0/1)	53,593	0.984	0.126	0.000	1.000	1.000	1.000	1.000
Panel B: Technical Debt								
Ln(Technical Debt in \$)	53,593	12.206	2.679	4.237	10.683	12.546	14.128	19.070
Technical Debt per Line of Code	53,593	9.245	14.400	0.001	3.395	6.191	10.692	691.576
Technical Debt Velocity	45,140	5.949	47.976	0.000	1.193	3.175	5.612	5,417.343
Panel C: Code Quality								
Ln(Lines of Code)	53,593	10.931	2.420	0.693	9.656	11.189	12.528	16.840
Ln(Open Issues)	53,593	6.852	2.455	0.693	5.357	7.080	8.630	13.754
Ln(Cognitive Complexity)	53,593	8.447	2.915	0.000	7.120	8.910	10.410	15.209
Ln(New Security Issues - High)	53,593	0.083	0.350	0.000	0.000	0.000	0.000	5.069
Total Security Hotspots	53,593	0.253	2.347	0.000	0.000	0.000	0.000	158.000
Panel D: Firm Characteristics								
Founding Year	37,884	2,015.075	7.628	1,951.000	2,013.000	2,017.000	2,020.000	2,024.000

Note: This table presents summary statistics for the firm-month panel used in the correlational analysis. The sample includes up to 53,593 firm-month observations; headcount variables have fewer observations due to missing LinkedIn data. Detailed variable definitions are provided in the appendix.

3 Empirical Framework and Results

A central challenge in linking cybersecurity vulnerabilities to firm outcomes is omitted variable bias. For instance, firms with better management may invest more in security and engineering practices and also grow faster for reasons unrelated to cybersecurity. As a result, simple correlations between vulnerability exposure and firm performance need not reflect causal effects.

We address this concern by exploiting public discoveries of vulnerabilities in third-party software dependencies used by firms in our sample. Once a vulnerability is discovered, any firm that happens to rely on an affected package version is exposed regardless of its own engineering quality or business strategy. Using our monthly codebase snapshots, we observe the exact dependency version in use at the time of discovery, which lets us assign exposure precisely even when firms do not publicly report the incident.

We focus on discoveries of critical-severity vulnerabilities, defined as vulnerabilities with CVSS scores of 9 or above. Appendix Table 16 defines the CVSS measure and our critical-severity threshold. In our sample, we identify 88 unique critical-severity vulnerability events affecting 121 firms through their third-party dependencies. These shocks are not infrequent: over our sample period, approximately one in five firms experiences at least one critical dependency vulnerability, and the 88 events cluster into 38 distinct cohorts used in our stacked design. Appendix Figure 13 shows that the temporal

distribution of these cohort events broadly mirrors the rising trend in critical vulnerability discoveries recorded in the GitHub Advisory Database, suggesting that our sample of shocks is representative of the general population of critical disclosures. Each shock requires the affected firm to either upgrade to a patched version of the dependency (which may introduce compatibility issues with existing code), downgrade to a pre-vulnerability version (potentially losing functionality), or refactor the codebase to remove the dependency entirely. The scope of this remediation effort depends on how deeply the vulnerable package is integrated into the firm’s product, but even straightforward dependency upgrades frequently trigger cascading changes across the codebase. These events are economically salient and difficult to ignore: even absent a realized breach, firms must devote engineering resources to remediation.

We estimate treatment effects using a stacked difference-in-differences (DiD) framework. We adopt this approach to address the staggered timing of vulnerability disclosures, where treatment occurs at different times across firms. A large body of econometric research demonstrates that, in the presence of dynamic treatment effects, standard two-way fixed effects (TWFE) estimators can be biased because they implicitly use already-treated units as controls for later-treated units, which can yield misleading inferences and, in some cases, produce estimates with the wrong sign ([Callaway and Sant’Anna, 2021](#); [Sun and Abraham, 2021](#)). Although this issue has been emphasized more recently in the econometrics literature, the underlying concern and cohort-based solutions were already recognized in earlier applied work ([Gormley and Matsa, 2011, 2016](#); [Baker et al., 2022](#)). Our stacked design circumvents this issue by constructing a “clean” counterfactual, ensuring that treated firms are compared strictly to not-yet-treated controls within the same calendar window ([Gormley and Matsa, 2011](#)). Identification requires two assumptions: (1) parallel trends—in the absence of treatment, treated and control firms would have followed parallel outcome paths; and (2) no anticipation—firms do not adjust behavior before the public discovery of the vulnerability. The second assumption is supported by the nature of our shocks: vulnerability discoveries are made by independent security researchers at unpredictable times, and firms have no advance knowledge of when a vulnerability in their dependency will be publicly disclosed.

To implement the stacked design, we match each critical-severity discovery to affected firms using the dependency–version mapping described in Section 2.5. Starting from our universe of 648 firms, we apply four sample restrictions. First, we restrict to firms that commercialize a single repository (their core product), ensuring that treatment status is unambiguous; this excludes firms whose exposure might be diluted across multiple products. Second, we require that firms have non-missing headcount data from LinkedIn during the event window, which drops firms for which we cannot observe the outcome variable. Third, we require that control firms have not experienced any critical dependency vulnerability during the 24-month event window (± 12 months around each shock), ensuring that the counterfactual is constructed from “clean” controls who are not simultaneously responding to their own vulnerability shocks. Fourth, we require that firms have valid SonarQube static analysis results

with positive lines of code, which is necessary for the technical debt and code quality outcomes that form the core of our mechanism analysis. Treated firms are those whose codebase contained the affected dependency version at the time of discovery. The resulting stacked dataset contains 284 firms (117 treated; 167 control) across 38 cohorts and 114,393 firm-by-cohort observations. Table 4 reports summary statistics for the stacked dataset.

Table 4: Summary Statistics - Stacked Difference-in-Differences Sample

Variable	Distribution							
	N	Mean	SD	Min	P25	Median	P75	Max
Panel A: Outcome Variables								
Ln(Headcount)	114,393	3.331	2.041	0.000	1.946	3.091	4.564	11.925
Funding Round (0/1)	114,393	0.024	0.152	0.000	0.000	0.000	0.000	1.000
Acquisition (0/1)	114,393	0.053	0.223	0.000	0.000	0.000	0.000	1.000
Is Alive (0/1)	114,393	0.983	0.128	0.000	1.000	1.000	1.000	1.000
Panel B: Technical Debt								
Ln(Technical Debt)	114,393	12.071	2.682	4.642	10.128	12.149	14.056	18.584
Panel C: Code Quality								
Ln(Lines of Code)	114,393	10.404	2.382	1.386	9.022	10.511	12.124	14.934
Ln(Open Issues)	114,393	6.677	2.499	0.693	4.860	6.782	8.521	13.291

Note: This table presents summary statistics for the stacked difference-in-differences sample. The sample includes 284 firms (117 treated, 167 control) across 38 cohorts. Detailed variable definitions are provided in the appendix.

Before turning to estimation, we assess the comparability of treated and control firms. Table 5 reports pre-treatment means of key observables for treated and control firms at $t = -1$ in the stacked DiD sample. Technical debt, security hotspots, open issues, high-severity security issues per thousand lines of code, bugs per thousand lines of code, duplicated lines, and funding rates are all well-balanced across groups, with normalized differences well below the 0.25 threshold recommended by [Imbens and Rubin \(2015\)](#). Treated firms display somewhat lower pre-treatment cognitive complexity (normalized difference of -0.24) and fewer critical violations (-0.23), indicating less complex, more maintainable codebases. Importantly, this direction of imbalance works *against* our findings: treated firms began with arguably better code quality, making the post-shock increase in technical debt and complexity a conservative estimate. All level differences are absorbed by firm-by-cohort fixed effects. Treated firms are somewhat smaller as measured by headcount (normalized difference of -0.42); while this level imbalance does not threaten identification given the within-firm design, it motivates our inclusion of lines of code as a size control. Moreover, smaller firms arguably have higher baseline headcount growth rates—a firm with 20 employees can achieve 20% year-over-year growth by hiring just four people—so any negative treatment effect on headcount growth is identified against a counterfactual of faster organic growth, further biasing against finding an effect. The full sample attrition funnel is reported in Appendix Table 26.

Table 5: Pre-Treatment Balance: Treated vs. Control Firms

	Treated	Control	Diff.	<i>p</i> -value	Norm. Diff.
Ln(Technical Debt)	12.054	12.036	0.017	0.87	0.007
Ln(Cognitive Complexity)	7.891	8.571	-0.680	<0.01	-0.238
Ln(Headcount)	2.626	3.419	-0.793	<0.01	-0.420
Ln(Lines of Code)	10.259	10.426	-0.167	0.07	-0.072
Ln(Security Hotspots)	3.657	3.712	-0.055	0.51	-0.025
Ln(Open Issues)	6.694	6.635	0.058	0.56	0.024
Ln(Critical Violations)	5.289	5.927	-0.639	<0.01	-0.230
Vulnerabilities / kLoC	0.101	0.560	-0.459	<0.01	-0.196
Security Issues, High / kLoC	0.025	0.017	0.007	0.24	0.056
Bugs / kLoC	3.228	2.681	0.548	0.16	0.058
Duplicated Lines	47,347	46,693	654	0.94	0.004
Funding Round	0.020	0.021	-0.001	0.88	-0.006
<i>N</i> (firm×cohort obs.)	696	4377			

Note: This table reports pre-treatment means at $t = -1$ (one month before vulnerability discovery) for treated and control firms in the stacked DiD sample. The normalized difference follows [Imbens and Rubin \(2015\)](#): $(\bar{X}_T - \bar{X}_C) / \sqrt{(s_T^2 + s_C^2)/2}$, where $\bar{X}_T$ and $\bar{X}_C$ are the sample means, and s_T^2 and s_C^2 are the sample variances for treated and control groups respectively. Values below 0.25 in absolute value suggest adequate balance; bold entries exceed this threshold. The *p*-value is from Welch's *t*-test allowing for unequal variances.

We estimate the following stacked difference-in-differences specification:

$$\ln(\text{Headcount})_{i,c,t} = \beta \cdot \text{did}_{i,c,t} + \gamma' X_{i,c,t} + \delta_{c,t} + \chi_{c,i} + \varepsilon_{i,c,t} \quad (2)$$

The dependent variable, $\ln(\text{Headcount})_{i,c,t}$, is the log headcount of firm i in cohort c at event time t . The treatment indicator $\text{did}_{i,c,t}$ equals $\text{Treated}_{i,c} \times \text{Post}_{c,t}$, where $\text{Post}_{c,t}$ indicates periods strictly after the disclosure month within cohort c (i.e., $t \geq 1$; the discovery month itself is coded as pre-period since exposure is partial during that month). We include time-varying firm and project controls $X_{i,c,t}$, firm-by-cohort fixed effects $\chi_{c,i}$, and event-time-by-cohort fixed effects $\delta_{c,t}$, so identifying variation comes from within-cohort changes in treated firms relative to not-yet-treated controls at the same event time. Standard errors are clustered at the firm level, which accounts for serial correlation within firm and for the fact that control firms may appear in multiple cohorts, inducing mechanical correlation across cohorts. Appendix Table 24 shows that results are robust to alternative clustering structures, including two-way clustering at the firm and cohort level.

Before presenting our estimates, we briefly discuss the expected direction of the effect on headcount. The sign is not obvious *a priori*. On one hand, firms could respond to vulnerability shocks by

increasing headcount—hiring additional engineers to handle remediation alongside ongoing product development. On the other hand, several mechanisms could produce a *decline* in headcount growth. First, vulnerability remediation diverts existing engineering resources from feature development and growth-oriented activities, potentially slowing the firm’s product roadmap and reducing its need for new hires. Second, the operational disruption and perceived technical risk may deter prospective employees or investors, making it harder to fund expansion. Third, for small firms operating with lean teams, the shock may force a reallocation of managerial attention and capital toward “firefighting” rather than scaling, reducing the pace of hiring. Given these competing forces, the net effect is an empirical question.

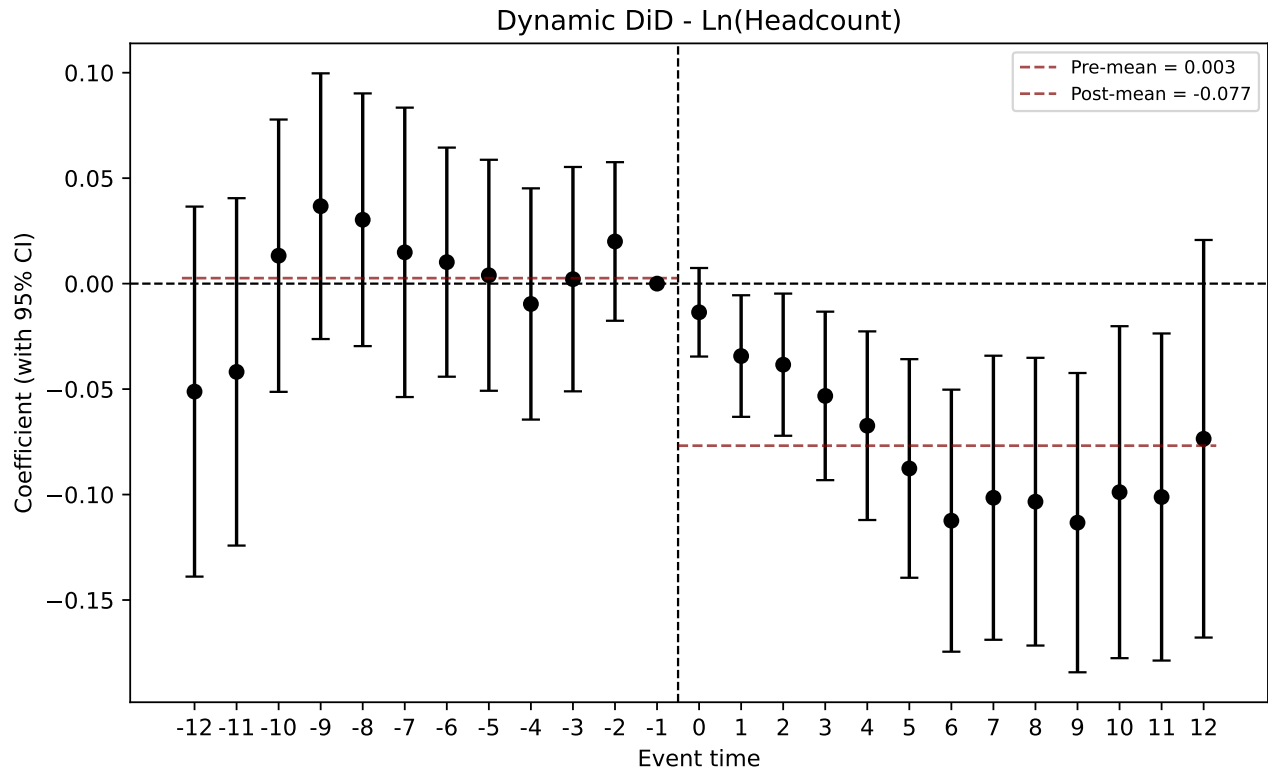


Figure 2: Cybersecurity Vulnerabilities and Firm Growth (Event Time in Months)

Table 6: Difference-in-Differences Estimates for Headcount

Dep Var: Ln(Headcount)	(1)	(2)	(3)	(4)
Treated $\times$ Post	-0.081* (0.042)	-0.082* (0.042)	-0.081* (0.042)	-0.087** (0.044)
# Lines of Code		0.000 (0.000)	0.000 (0.000)	0.000 (0.000)
Firm $\times$ Cohort FE	Yes	Yes	Yes	Yes
Relative Month $\times$ Cohort FE	Yes	Yes	Yes	Yes
Language $\times$ Cohort FE			Yes	
Firm Type $\times$ Month FE				Yes
R^2	0.981	0.981	0.982	0.982
# Obs	114,393	114,393	114,393	114,393

Note: This table presents stacked difference-in-differences estimates of the effect of critical vulnerability discoveries on firm headcount over a ± 12 month event window. The dependent variable is the natural log of firm headcount (from LinkedIn). The sample consists of 284 firms (117 treated, 167 control) across 38 cohorts. Treated firms are those whose codebase contained an affected dependency version at the time of a critical-severity disclosure ($CVSS \geq 9$). Controls are firms with no critical dependency vulnerability during the 24-month event window. Column (1) is the baseline specification with firm-by-cohort and event-time-by-cohort fixed effects only. Column (2) adds lines of code as a time-varying control for project scale. Column (3) further absorbs programming-language-by-cohort fixed effects. Column (4) replaces language fixed effects with firm-type-by-calendar-month fixed effects, where firm types are five product categories (Infrastructure, Developer Tools, Security, Data & Analytics, Business Applications) derived from LLM classification of each firm's core product (Table 1). Standard errors in parentheses are clustered at the firm level. * $p < 0.1$, ** $p < 0.05$, *** $p < 0.01$.

Figure 2 shows the monthly dynamic treatment effects on headcount growth around the discovery month ². One concern is that the positive pre-trend could reflect mean-reversion in headcount growth among smaller firms (treated firms are somewhat smaller; see Table 5). However, this interpretation is inconsistent with the pattern in Figure 2, where the pre-period coefficients are approximately flat rather than declining, and the post-period decline is discontinuous at $t = 0$ —consistent with a discrete shock rather than gradual reversion. Leave-one-cohort-out analysis confirms that the result is stable across all 38 cohorts (coefficient range: -0.066 to -0.091 ; see Section 3.4). After discovery, treated firms experience a persistent decline in growth velocity relative to controls.

Table 6 reports the corresponding regression estimates. Column (1) is the baseline specification with only firm-by-cohort and event-time-by-cohort fixed effects (no additional time-varying controls) and yields a coefficient of -0.081 (significant at the 10% level). Column (2) adds controls for project scale measured by lines of code; the estimate remains stable at -0.082 (significant at the 10% level). Column (3) further absorbs programming-language-by-cohort fixed effects to address compositional differences across treatment arms within each cohort; the estimate is virtually unchanged at -0.081 (significant at the 10% level).³ Column (4) replaces language fixed effects with firm-type-by-calendar-month fixed effects, absorbing industry-specific time trends; the estimate of -0.087 is significant at the 5% level. The stability of the coefficient across all four specifications suggests that the headcount effect is not driven by language-ecosystem or industry-segment confounders. Under our preferred specification, the coefficient of -0.081 indicates that treated firms’ headcount grows 8.1 percentage points slower than controls, implying $\exp(-0.081) - 1 \approx 7.8\%$ lower headcount levels relative to the counterfactual. To benchmark this magnitude, control firms in our sample exhibit median annual headcount growth of 36.3% (12-month $\Delta \ln(\text{HC}) = 0.310$); the treatment effect thus eliminates roughly one-fifth of a year’s growth ($7.8/36.3 \approx 0.21$). For the median firm in our sample (19 employees), this translates to approximately 1.5 fewer employees over the post-vulnerability year—reducing expected growth from roughly 6.9 to 5.4 new employees—a meaningful reduction for small teams operating with limited engineering capacity.

We next examine whether these shocks spill over into firms’ ability to secure financing. Given the operational strain imposed by vulnerability remediation, we hypothesize that affected firms may be less likely to raise capital in the immediate aftermath of a shock. This could occur through multiple channels: investors may perceive heightened operational risk, firms may delay fundraising to focus engineering resources on remediation, or the diversion of resources from feature development may reduce firms’ ability to demonstrate progress toward milestones that typically precede funding events.

Our funding outcome is an indicator for whether a firm raises a financing round during the event

²The pre-discovery coefficients are economically small, though a formal joint test on the eleven monthly pre-treatment coefficients rejects the null of exact parallel pre-trends ($F(11) = 30.1$, $p = 0.002$).

³For the headcount specification, we use language-by-cohort fixed effects in Column (3) rather than language-by-month, because headcount is available only for a subset of firms and the language-by-month cells can become sparse. All other outcome tables use language-by-month fixed effects in Column (3).

window (seed, early-stage VC, late-stage VC, PE growth/expansion, accelerator/incubator, or angel). Figure 3 plots the event-time estimates, where the underlying monthly data are aggregated to quarterly coefficients for clarity. The estimates are noisy throughout, with wide confidence intervals spanning zero in every period.

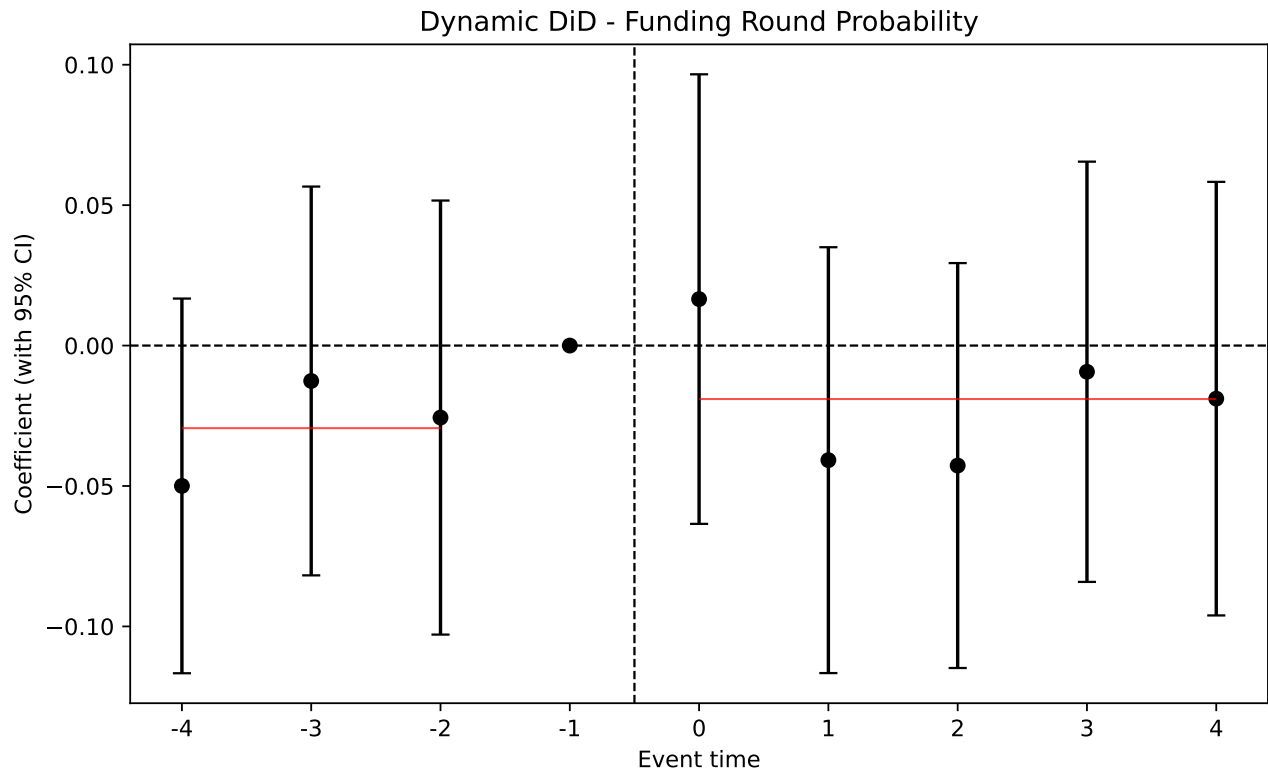


Figure 3: Cybersecurity Vulnerabilities and Funding Rounds (Event Time in Quarters)

Table 7: Difference-in-Differences Estimates for Funding

Dep Var: Funding Round (Pre/Post)	(1)	(2)	(3)	(4)
Treated $\times$ Post	0.028 (0.063)	0.010 (0.064)	0.010 (0.064)	0.029 (0.062)
# Lines of Code			-0.000 (0.000)	-0.000 (0.000)
Firm $\times$ Cohort FE	Yes	Yes	Yes	Yes
Relative Month $\times$ Cohort FE	Yes	Yes	Yes	Yes
Language $\times$ Month FE		Yes	Yes	
Firm Type $\times$ Month FE				Yes
R^2	0.667	0.683	0.683	0.672
# Obs	114,393	114,393	114,393	114,393

Note: This table presents stacked difference-in-differences estimates of the effect of critical vulnerability discoveries on firm funding over a ± 12 month event window. The dependent variable is an indicator equal to one if the firm received a funding round (from PitchBook) in a given month. The sample and treatment assignment follow Table 6. Column (1) includes firm-by-cohort and event-time-by-cohort fixed effects only. Column (2) adds language-by-month fixed effects. Column (3) further controls for project scale (lines of code). Column (4) replaces language fixed effects with firm-type-by-calendar-month fixed effects. Standard errors in parentheses are clustered at the firm level. * $p < 0.1$, ** $p < 0.05$, *** $p < 0.01$.

Table 7 reports the static estimates. Point estimates are positive but small and statistically insignificant across all specifications (0.028 in column (1), declining to 0.010 in columns (2) and (3), and 0.029 in column (4)), precluding strong conclusions about the effect of vulnerability discoveries on financing. Column (1) includes only firm-by-cohort and event-time-by-cohort fixed effects, while columns (2) and (3) add language-by-month fixed effects to absorb time-varying funding conditions common to firms built on similar technology stacks, along with controls for project scale. Column (4) uses firm-type-by-month fixed effects as an alternative saturated structure, yielding a similarly insignificant estimate.

We also examine the probability of being acquired. Our outcome is an indicator for whether a firm is acquired (merger/acquisition or buyout/LBO) during the post-vulnerability period. Table 8 shows a statistically significant decline in acquisition probability across specifications. In the baseline model (column (1)), treated firms are 1.5 percentage points less likely to be acquired (significant at the 1% level). With language-by-month fixed effects and controls for project scale (columns (2) and (3)), the estimate is -0.017 (significant at the 5% level).

Table 8: Difference-in-Differences Estimates for Acquisition

Dep Var: Acquisition (Pre/Post)	(1)	(2)	(3)	(4)
Treated $\times$ Post	-0.015*** (0.006)	-0.017** (0.007)	-0.017** (0.007)	-0.016** (0.006)
# Lines of Code			-0.000 (0.000)	-0.000 (0.000)
Firm $\times$ Cohort FE	Yes	Yes	Yes	Yes
Relative Month $\times$ Cohort FE	Yes	Yes	Yes	Yes
Language $\times$ Month FE		Yes	Yes	
Firm Type $\times$ Month FE				Yes
R^2	0.950	0.952	0.952	0.950
# Obs	114,393	114,393	114,393	114,393

Note: This table presents stacked difference-in-differences estimates of the effect of critical vulnerability discoveries on acquisition probability over a ± 12 month event window. The dependent variable is an indicator equal to one if the firm was acquired (merger/acquisition or buyout/LBO, from PitchBook) during the post-vulnerability period. The sample and treatment assignment follow Table 6. Column (1) includes firm-by-cohort and event-time-by-cohort fixed effects only. Column (2) adds language-by-month fixed effects. Column (3) further controls for project scale (lines of code). Column (4) replaces language fixed effects with firm-type-by-calendar-month fixed effects. Standard errors in parentheses are clustered at the firm level. * $p < 0.1$, ** $p < 0.05$, *** $p < 0.01$.

One interpretation is that critical vulnerabilities reduce firms' attractiveness as acquisition targets by increasing perceived technical risk and the expected integration burden, particularly if remediation generates technical debt or slows product progress. Relative to the unconditional acquisition rate of 5.3%, the baseline coefficient of -0.015 represents a 28% relative decline in acquisition probability (32% using the -0.017 estimate from column 3), a meaningful reduction in exit opportunities for firms in this segment. The estimate is stable under firm-type-by-month fixed effects (column 4: -0.016 , significant at the 5% level), ruling out that the result is driven by industry-segment-specific acquisition trends.

Table 9: Difference-in-Differences Estimates for Survival

Dep Var: Is Alive (Pre/Post)	(1)	(2)	(3)	(4)
Treated $\times$ Post	0.002 (0.001)	0.002 (0.001)	0.002 (0.001)	0.002 (0.002)
Ln(Open Issues)		0.001 (0.000)	0.000 (0.000)	0.000 (0.000)
Ln(Lines of Code)			0.000 (0.000)	0.001 (0.001)
Firm $\times$ Cohort FE	Yes	Yes	Yes	Yes
Relative Month $\times$ Cohort FE	Yes	Yes	Yes	Yes
Firm Type $\times$ Month FE				Yes
R^2	0.951	0.951	0.951	0.952
# Obs	114,393	114,393	114,393	114,393

Note: This table presents stacked difference-in-differences estimates of the effect of critical vulnerability discoveries on firm survival over a ± 12 month event window. The dependent variable is an indicator equal to one if the firm is alive (not closed or defunct per PitchBook) in a given month. The sample and treatment assignment follow Table 6. Column (1) is the baseline specification with firm-by-cohort and event-time-by-cohort fixed effects. Column (2) adds the natural log of open issues as a time-varying control. Column (3) further controls for project scale (natural log of lines of code). Column (4) adds firm-type-by-calendar-month fixed effects. Standard errors in parentheses are clustered at the firm level. * $p < 0.1$, ** $p < 0.05$, *** $p < 0.01$.

Finally, Table 9 examines survival, measured by an indicator for whether the firm remains operational (not out of business according to PitchBook) during the post-vulnerability period. The estimates are small (0.002) and statistically insignificant across all specifications. Given the low failure rate in our sample and the relatively short event window, we do not find evidence that critical vulnerability shocks materially affect short-run survival, even though they affect growth and acquisition outcomes.

Taken together, the evidence suggests that critical vulnerability shocks impose real organizational costs. Treated firms experience slower headcount growth and are less likely to exit via acquisition, while we find no detectable effect on short-run survival. We do not find statistically significant effects on funding outcomes. The combination of significant headcount and acquisition effects with null funding and survival results paints a picture of firms that survive but with stunted growth rather than

firm death. The shock degrades the firm’s core intangible asset (codebase quality) without triggering outright financial distress, consistent with firms that continue to operate but reallocate engineering resources from innovation to maintenance. In effect, affected firms may be “running just to stand still,” devoting effort to patching and remediation rather than building new capabilities, which in turn constrains their ability to scale or attract acquirers.

3.1 Impact on Technical Debt

Having established that critical vulnerability discoveries affect firm-level outcomes, we next investigate the mechanisms through which these transient shocks may translate into persistent constraints on firm performance. Given our unique capacity to quantify firm-level software liabilities, we examine *technical debt* as both an outcome and a potential channel linking cybersecurity shocks to long-term organizational costs.

Following our previous discussion, technical debt accumulates when suboptimal engineering practices are adopted to meet objectives more quickly. In this context, the urgency arises from the need to remediate a publicly disclosed vulnerability. Given the critical severity of these vulnerabilities, firms cannot simply ignore them. Upon a vulnerability announcement, a firm faces two main choices. It can either modify its code to remove the affected package entirely, a process that requires substantial engineering effort to rewrite dependent components, or it can switch to a different package version that is not affected. However, this alternative is also laborious: vulnerabilities often span multiple versions prior to discovery, meaning a downgrade may entail reverting to an outdated version that lacks essential functionality the firm relies on. Conversely, upgrading to the latest patched version can create compatibility issues with existing software, requiring additional developer effort to restore stability.

We aim to examine the extent to which such shocks increase the technical debt principal of affected firms. This is particularly important, as technical debt is widely regarded as a significant software-related liability in the computer science literature. We measure firm-level TD around our ± 12 month event window following the process outlined in Section 2.7. We then implement a similar stacked difference-in-differences specification to estimate the impact of cybersecurity shocks on its accumulation. Formally, we estimate the following model:

$$\ln(\text{Technical Debt in USD})_{i,c,t} = \beta \cdot did_{i,c,t} + \gamma' X_{i,c,t} + \delta_{c,t} + \chi_{c,i} + \varepsilon_{i,c,t}$$

The dependent variable $\ln(\text{Technical Debt in USD})_{i,c,t}$ denotes the technical debt principal of firm i in cohort c at time t . The treatment indicator $did_{i,c,t}$ captures whether firm i in cohort c is affected by a critical vulnerability by time t . We include a vector of time-varying firm and project

controls $X_{i,c,t}$, firm-cohort fixed effects $\chi_{c,i}$, and time-cohort fixed effects $\delta_{c,t}$. The error term is denoted by $\varepsilon_{i,c,t}$.

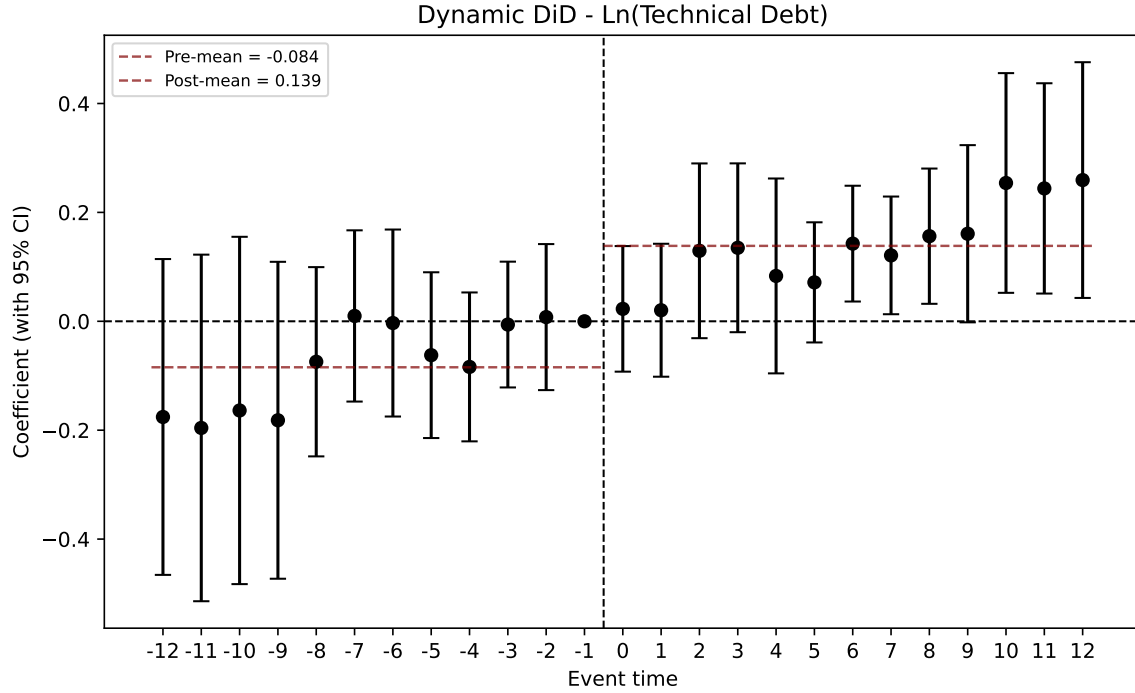


Figure 4: Difference in Differences - Technical Debt (Event Time in Months)

Table 10: Difference-in-Differences Estimates for Ln(Technical Debt in USD)

Dep Var: Ln(Technical Debt in USD)	(1)	(2)	(3)	(4)
Treated $\times$ Post	0.204** (0.095)	0.184** (0.089)	0.173 (0.112)	0.157* (0.088)
# Lines of Code		0.000*** (0.000)	0.000*** (0.000)	0.000*** (0.000)
Firm $\times$ Cohort FE	Yes	Yes	Yes	Yes
Relative Month $\times$ Cohort FE	Yes	Yes	Yes	Yes
Language $\times$ Month FE			Yes	
Firm Type $\times$ Month FE				Yes
R^2	0.937	0.944	0.961	0.946
# Obs	114,393	114,393	114,393	114,393

Note: This table presents stacked difference-in-differences estimates of the effect of critical vulnerability discoveries on technical debt over a ± 12 month event window. The dependent variable is the natural log of technical debt in USD, measured by SonarQube static analysis and calibrated to remediation cost benchmarks. The sample and treatment assignment follow Table 6. Column (1) is the baseline specification with firm-by-cohort and event-time-by-cohort fixed effects. Column (2) adds lines of code as a time-varying control for project scale. Column (3) further absorbs language-by-month fixed effects. Column (4) replaces language fixed effects with firm-type-by-calendar-month fixed effects. Standard errors in parentheses are clustered at the firm level. * $p < 0.1$, ** $p < 0.05$, *** $p < 0.01$.

We find that technical debt is substantially higher for treated firms relative to controls. Table 10 presents the results. The baseline specification in column (1) yields a coefficient of 0.204, significant at the 5% level, indicating that treated firms accumulate technical debt 20.4 percentage points faster than controls—equivalent to $\exp(0.204) - 1 \approx 22.6\%$ higher levels. When we control for project size in column (2), the effect remains significant at the 5% level, with treated firms accumulating technical debt 18.4 percentage points faster (20.2% higher levels). In column (3), we add language-by-month fixed effects, which absorb time-varying conditions common to codebases built on the same programming language (e.g., ecosystem-wide tooling changes or library updates that differentially affect technical debt). The point estimate declines to 0.173 and is not statistically significant at conventional levels. This attenuation suggests that part of the baseline effect may operate through language-ecosystem-specific channels—for instance, if vulnerability shocks disproportionately affect certain language ecosystems during particular periods. However, the sign and approximate magnitude of the coefficient remain consistent with columns (1) and (2), and the loss of precision may partly reflect the demanding nature of the saturated fixed-effects structure. Column (4) uses firm-type-by-month fixed effects instead; the coefficient of 0.157 is significant at the 10% level, confirming that the result is not driven by industry-segment-specific trends in technical debt. Overall, the pattern across specifications is consistent with the notion that technical debt accumulates as firms prioritize short-term objectives. In our context, a key objective is the rapid mitigation of exposure to a newly disclosed vulnerability, which may prompt engineers to adopt fast but suboptimal coding solutions. To benchmark, control firms exhibit median annual technical debt growth of 7.0% (12-month $\Delta \ln(\text{TD}) = 0.068$); the treatment effect of 0.204 log points is approximately three times this baseline annual rate. In dollar terms, the average treated firm enters the event window carrying \$219,000 in technical debt (geometric mean at $t = 0$). At the counterfactual growth rate, such a firm would add roughly \$15,300 in new technical debt over the post-shock year; the treatment effect adds approximately \$30,700 on top of this—equivalent to roughly two additional years of counterfactual technical debt growth compressed into a single post-shock year. At an hourly engineering wage of \$55, this represents approximately 558 hours, or roughly 3.5 engineer-months, of additional remediation work.

These results provide suggestive evidence that critical vulnerability shocks increase software-related liabilities for firms. We next show that cognitive complexity also increases following these shocks, and then relate complexity to longer-run firm growth.

3.2 Additional Evidence: Cognitive Complexity and Firm Growth

We complement the technical-debt results by examining cognitive complexity, a maintainability metric derived from SonarQube. First, we estimate the stacked difference-in-differences specification using $\ln(\text{Cognitive Complexity})$ of the codebase as the dependent variable. Unlike simple measures such as lines of code, cognitive complexity captures the mental effort required to comprehend control flow,

nested conditions, and non-linear logic (see Section 2.7 and Appendix A.5 for measurement details). Higher cognitive complexity indicates code that is harder to modify, debug, and extend—precisely the type of code that accumulates when teams prioritize speed over maintainability, such as when responding to critical security vulnerabilities. Table 11 shows that cognitive complexity grows 19–21 percentage points faster in treated firms relative to controls in the post-vulnerability period (19–23% higher levels in baseline specifications), consistent with time-sensitive remediation.

Table 11: Difference-in-Differences Estimates for Ln(Cognitive Complexity)

Dep Var: Ln(Cognitive Complexity)	(1)	(2)	(3)	(4)
Treated × Post	0.207* (0.107)	0.191* (0.101)	0.184* (0.106)	0.160 (0.100)
# Lines of Code		0.000*** (0.000)	0.000*** (0.000)	0.000*** (0.000)
Firm × Cohort FE	Yes	Yes	Yes	Yes
Relative Month × Cohort FE	Yes	Yes	Yes	Yes
Language × Month FE			Yes	
Firm Type × Month FE				Yes
R^2	0.940	0.943	0.968	0.945
# Obs	114,393	114,393	114,393	114,393

Note: This table presents stacked difference-in-differences estimates of the effect of critical vulnerability discoveries on cognitive complexity over a ± 12 month event window. The dependent variable is the natural log of cognitive complexity, winsorized at the 1st and 99th percentiles, a SonarQube metric that quantifies the mental effort required to comprehend control flow and nested logic (see Appendix A.5). The sample and treatment assignment follow Table 6. Column (1) is the baseline specification with firm-by-cohort and event-time-by-cohort fixed effects. Column (2) adds lines of code as a time-varying control for project scale. Column (3) further absorbs language-by-month fixed effects. Column (4) replaces language fixed effects with firm-type-by-calendar-month fixed effects. Standard errors in parentheses are clustered at the firm level. * $p < 0.1$, ** $p < 0.05$, *** $p < 0.01$.

We then provide correlational evidence on the relationship between cognitive complexity and longer-run firm growth, including specifications that control for new security issues.

We estimate the following model:

$$\text{Growth}_{t+24} = \beta \cdot \ln(\text{Cognitive Complexity})_{i,t} + \gamma' X_{i,t} + \chi_i + \delta_t + \varepsilon_{i,t} \quad (3)$$

The dependent variable Growth_{t+24} is the firm's headcount 24 months later relative to its headcount at time t . The main regressor is $\ln(\text{Cognitive Complexity})_{i,t}$, the natural logarithm of aggregate cognitive complexity across the firm's codebase. We include firm fixed effects χ_i , month

fixed effects δ_t , and controls for the dominant programming language. The error term is $\varepsilon_{i,t}$. This analysis relies on the firm-level monthly dataset rather than the cohort-based sample used in the stacked difference-in-differences specification.

Table 12: Cognitive Complexity and Firm Growth

Dep Var: $Growth_{t+24}$	(1)	(2)	(3)	(4)	(5)	(6)
Ln(Cognitive Complexity)	-0.045 (0.045)	-0.106* (0.055)	-0.103* (0.055)	-0.138** (0.063)	-0.137** (0.063)	-0.124* (0.064)
Ln(New Security Issues - High)					0.096 (0.114)	0.083 (0.112)
Total Security Hotspots						-0.000 (0.000)
Firm FE		Yes	Yes	Yes	Yes	Yes
Month FE			Yes	Yes	Yes	Yes
Language FE				Yes	Yes	Yes
R^2	0.002	0.421	0.422	0.426	0.426	0.426
# Obs	12,859	12,858	12,858	12,856	12,856	12,856

Note: This table presents OLS estimates of the correlational relationship between cognitive complexity and two-year-ahead headcount growth. The dependent variable is the firm's headcount 24 months later relative to its headcount at time t . The sample is the broader firm-level monthly panel (not the stacked DiD sample). Column (1) is a pooled OLS regression with no fixed effects. Columns (2)–(6) progressively add firm, month, and programming language fixed effects, as well as controls for the flow of new high-severity security issues (from SonarQube) and total security hotspots. Standard errors in parentheses are clustered at the firm level. * $p < 0.1$, ** $p < 0.05$, *** $p < 0.01$.

We find that a 10% increase in cognitive complexity is associated with a 1.18–1.32 percentage point reduction in two-year-ahead headcount growth (columns (4)–(6) of Table 12; calculated as $\ln(1.10) \times \hat{\beta}$). This effect is statistically significant at the 5% level and robust to the inclusion of firm, month, and programming language fixed effects. The coefficient remains significant at the 5% level when controlling for the flow of new high-severity security issues as detected by SonarQube (column (5)). When we further add total security hotspots (column (6)), the point estimate declines modestly to -0.124 and significance falls to the 10% level, though the magnitude remains economically meaningful and the loss of precision may partly reflect the demanding fixed-effects structure. This pattern suggests that the complexity channel operates largely independently of direct security concerns. While this relationship should not be interpreted causally, it is consistent with the notion that complex, hastily written code—such as emergency patches responding to newly discovered vulnerabilities—

may impose persistent constraints on organizational growth by diverting engineering resources from innovation to maintenance.

3.3 Commit-Level Evidence of Remediation

The preceding analysis establishes that vulnerability shocks accelerate the accumulation of technical debt and cognitive complexity. We now provide direct evidence that these effects arise through time-sensitive remediation efforts. Our argument is that vulnerability discoveries trigger elevated development activity that, due to time pressure, generates the suboptimal code patterns documented above. To test this mechanism, we analyze repository-level commit activity around vulnerability discoveries.

We first examine whether vulnerability discoveries trigger an immediate increase in development activity. Using daily-frequency specifications over narrow event windows, we estimate the treatment effect on raw commit counts. Figure 5 presents dynamic treatment effects for total commits in a ± 3 day window around vulnerability discovery. The coefficient on Treated $\times$ Post is positive and statistically significant at the 5% level, indicating that treated repositories in the firm-level DiD sample produce approximately 2.7 more commits than control repositories in the days immediately surrounding discovery. This effect attenuates in wider windows (± 7 days: 1.2 additional commits, significant at the 10% level; ± 14 days: 0.5 additional commits, not statistically significant), consistent with the increase in development activity being concentrated in the immediate aftermath of discovery.

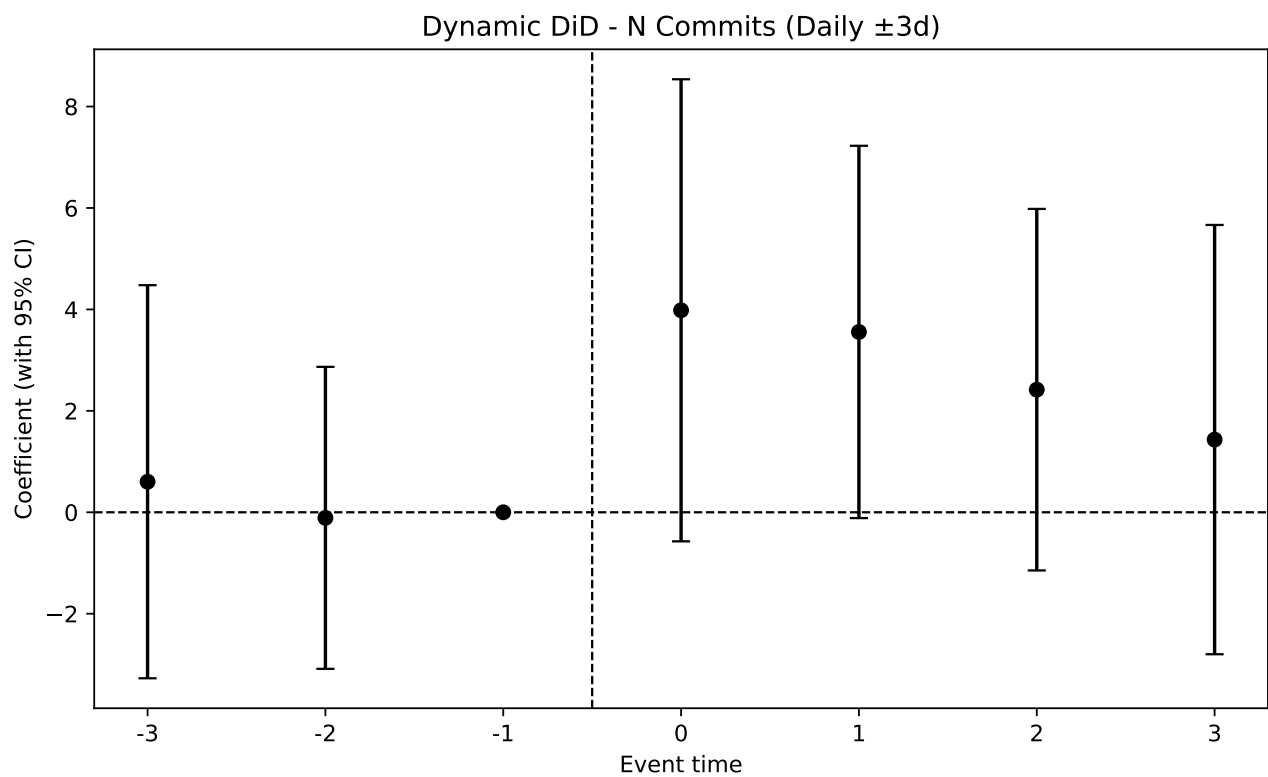


Figure 5: Immediate Commit Response to Vulnerability Discovery (± 3 Days)

Table 13 summarizes the static difference-in-differences estimates for commit intensity across event windows of varying width. The effect is largest and most precisely estimated in the narrowest window, declining monotonically as the window expands.

Table 13: Daily Difference-in-Differences Estimates for Commit Intensity

	± 3 Days	± 7 Days	± 14 Days
<i>Panel A: All Analyzed Repositories</i>			
Commits (Count)	1.613** (0.696)	0.468 (0.357)	0.247 (0.266)
# Obs	45,858	75,288	128,700
<i>Panel B: Stacked DiD Repositories</i>			
Commits (Count)	2.700** (1.141)	1.192* (0.635)	0.507 (0.386)
# Obs	13,787	22,823	39,359
Repo $\times$ Cohort FE	Yes	Yes	Yes
Relative Day $\times$ Cohort FE	Yes	Yes	Yes

Note: This table presents stacked difference-in-differences estimates of the effect of critical vulnerability discoveries on daily commit intensity. The dependent variable is the raw number of commits per repository-day. Panel A uses all repositories with SonarQube data; Panel B restricts the sample to repositories included in the firm-level stacked DiD analysis. Standard errors in parentheses are clustered at the repository level. * $p < 0.1$, ** $p < 0.05$, *** $p < 0.01$.

The commit intensity results in Table 13 show that treated repositories produce more commits, but they do not distinguish routine development from remediation-specific work. To isolate the remediation channel, we classify each commit as *vulnerability-resolution related* if its message contains any of the terms *bump*, *upgrade*, *CVE-**, *CVSS*, *vulnerability*, *security-fix*, or *security-patch*. These terms capture version bumps and security-specific patches—the direct mechanical response to a discovered vulnerability—while excluding general development language. Approximately 3.5% of all commits in our sample contain at least one such term.

Table 14 reports the results. In Panel A, treated repositories exhibit a statistically significant increase in vulnerability-resolution language at the ± 7 and ± 14 day windows: 1.6 percentage points at ± 7 days ($p < 0.01$) and 1.1 percentage points at ± 14 days ($p < 0.05$). The ± 3 day estimate is positive but imprecise ($\hat{\beta} = 0.014$, $p = 0.17$), likely reflecting the narrow window’s limited statistical power. Panel B shows a similar pattern: the ± 3 day estimate is again imprecise ($\hat{\beta} = 0.017$, $p = 0.25$), while the effect strengthens at ± 14 days ($\hat{\beta} = 0.015$, $p < 0.05$). The ± 30 day column provides a falsification-style test: the coefficient attenuates to near zero and loses significance entirely ($\hat{\beta} = 0.0005$, $p = 0.87$ in Panel A; $\hat{\beta} = 0.004$, $p = 0.41$ in Panel B), confirming that the vulnerability-resolution spike is concentrated within the first two weeks and does not reflect a secular trend. Figure 6 presents the

daily dynamic coefficients for the ± 14 day window. Pre-discovery coefficients are noisy but generally centered near zero, with no individually significant pre-trend; in the post-period, point estimates shift upward, consistent with the significant static effect reported in the table. The sharp attenuation of the static coefficient at ± 30 days provides the clearest evidence of a short-lived remediation burst that dissipates within two weeks.

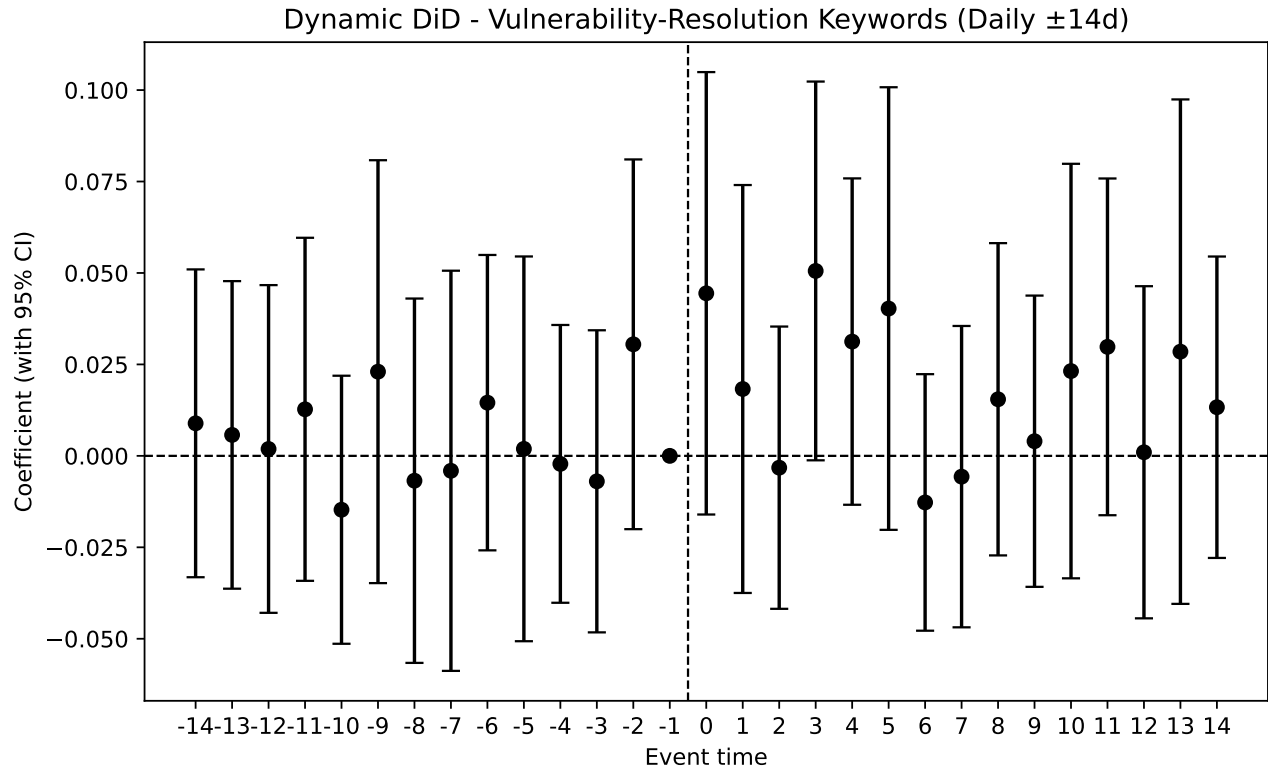


Figure 6: Cybersecurity Vulnerabilities and Vulnerability-Resolution Commit Activity (± 14 Days)

Table 14: Daily Difference-in-Differences Estimates for Vulnerability-Resolution Keywords per Commit

	± 3 Days	± 7 Days	± 14 Days	± 30 Days
<i>Panel A: All Analyzed Repositories</i>				
Vuln.-Res. Kw./Commit	0.0136 (0.0099)	0.0159*** (0.0059)	0.0105** (0.0043)	0.0005 (0.0030)
# Obs	45,858	75,288	128,700	231,182
<i>Panel B: Stacked DiD Repositories</i>				
Vuln.-Res. Kw./Commit	0.0174 (0.0152)	0.0149* (0.0087)	0.0147** (0.0059)	0.0036 (0.0044)
# Obs	13,787	22,823	39,359	70,560
Repo $\times$ Cohort FE	Yes	Yes	Yes	Yes
Relative Day $\times$ Cohort FE	Yes	Yes	Yes	Yes

Note: Reconstructed from committed commit messages and daily commit DiD cohorts. Keywords: *bump*, *upgrade*, *CVE-**, *CVSS*, *vulnerability*, *security-fix*, *security-patch*. Standard errors in parentheses are clustered at the repository level. * $p < 0.1$, ** $p < 0.05$, *** $p < 0.01$.

While the daily analysis captures the immediate response, vulnerability remediation extends beyond the first few days. Patching dependencies often triggers cascading changes: updating one library may break compatibility with others, requiring additional fixes; security patches may introduce new bugs that must be addressed; and previously deferred maintenance may become necessary when touching affected code. We therefore examine whether the *content* of commits shifts toward remediation-focused work over a longer horizon.

We construct a monthly commit panel by aggregating daily commit data and applying the same stacked difference-in-differences framework used in our firm-level analysis, now over a ± 12 month event window to capture the full arc of remediation.⁴ For the monthly analysis, we use the broader sample of all repositories with SonarQube data, comprising 464,145 repository-month observations across treated and clean control repositories.⁵ For each commit, we create a binary indicator equal to one if its message contains any fix-related keyword—specifically, terms such as *fix*, *bugfix*, *hotfix*, *patch*, *repair*, *resolve*, or *correct* and their inflected forms (e.g., *fixes*, *fixed*, *fixing*, *resolved*, *corrected*).

⁴Unlike the code-quality analysis, which uses only the final commit of each month to measure codebase state, the commit-activity analysis aggregates *all* commits within each month to measure development intensity and content.

⁵The daily commit analysis presents results for both the broader sample of all repositories with SonarQube data and the subset used in the firm-level stacked DiD (Table 13). The commit outcomes (keyword frequencies) are measured directly from git history and do not require SonarQube code-quality metrics.

The dependent variable, fix-related keywords per commit, is the monthly proportion of commits for which this indicator equals one, capturing the share of development activity devoted to remediation.

Figure 7 presents the monthly dynamic treatment effects for fix-related keywords per commit over the ± 12 month event window. The pre-discovery coefficients are close to zero and statistically insignificant, supporting the parallel trends assumption. Following discovery, we observe a persistent increase in fix-related commit language, with effects emerging in the first month post-discovery and remaining elevated through the event window. Table 15 reports the corresponding static estimates. In the broader sample of repositories with SonarQube data, treated repositories experience 3.2 percentage points more fix-related keywords per commit (significant at the 1% level; column 1). The effect is robust to restricting the sample to the same repositories used in the firm-level stacked DiD analysis ($\hat{\beta} = 0.019$, significant at the 5% level; column 2), though somewhat attenuated by the smaller sample size. Average commit message length also increases post-discovery (see Appendix Table 18), which could indicate that developers implement more complicated changes that require larger commit messages.

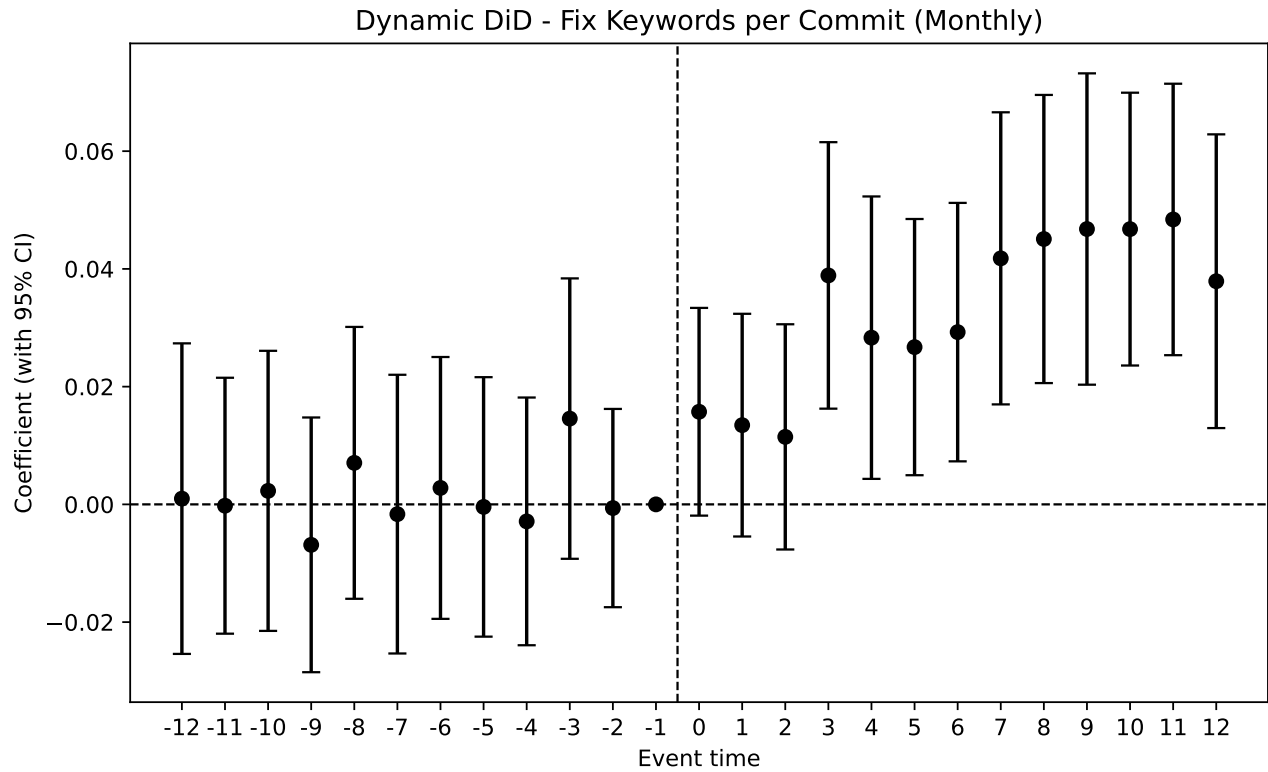


Figure 7: Cybersecurity Vulnerabilities and Fix-Related Commit Activity (± 12 Months)

Table 15: Monthly Difference-in-Differences Estimates for Fix-Related Keywords per Commit

	All Analyzed Repositories (1)	Stacked DiD Repositories (2)
Treated $\times$ Post	0.032*** (0.006)	0.019** (0.008)
Repo $\times$ Cohort FE	Yes	Yes
Relative Month $\times$ Cohort FE	Yes	Yes
R^2	0.494	0.621
# Obs	464,145	116,365

Note: This table presents stacked difference-in-differences estimates of the effect of critical vulnerability discoveries on fix-related keywords per commit over a ± 12 month event window. The dependent variable is the proportion of commits containing fix-related language (e.g., “fix,” “patch,” “hotfix”). Column (1) uses all repositories with SonarQube data; column (2) restricts to repositories in the firm-level stacked DiD sample. Standard errors in parentheses are clustered at the repository level. * $p < 0.1$, ** $p < 0.05$, *** $p < 0.01$.

These commit-level results are consistent with vulnerability discoveries triggering time-sensitive remediation efforts—conditions that the software engineering literature associates with the accumulation of technical debt and code complexity (Sculley et al., 2014). The pattern we document suggests that teams prioritize speed of response, potentially at the expense of code quality. Considering the results regarding technical debt and cognitive complexity, this evidence is suggestive of a channel in which rapid remediation of cybersecurity vulnerabilities contributes to lasting software liabilities, though we cannot rule out alternative explanations for the observed divergence in code quality.

Development Composition Shift. A key feature of technical debt as a liability is that remediation diverts engineering resources from innovation to maintenance, slowing the pace at which firms can develop new software features—the product capabilities that end users ultimately observe (Section 2.7). The results for fix-related commits document the maintenance side of this prediction; we now ask whether the composition of development activity shifts away from the creation of new features. We extend the dynamic analysis of code changes to four categories (fix, refactoring, feature, documentation) and estimate the event-study specification separately for each. The dependent variable in each panel is the share of commits in a given repository-month that contain at least one keyword from the relevant category, expressed as a percentage of total commits. Coefficients are therefore interpretable as percentage-point changes in the composition of development activity. Figure 8 presents the results for the four most informative categories.

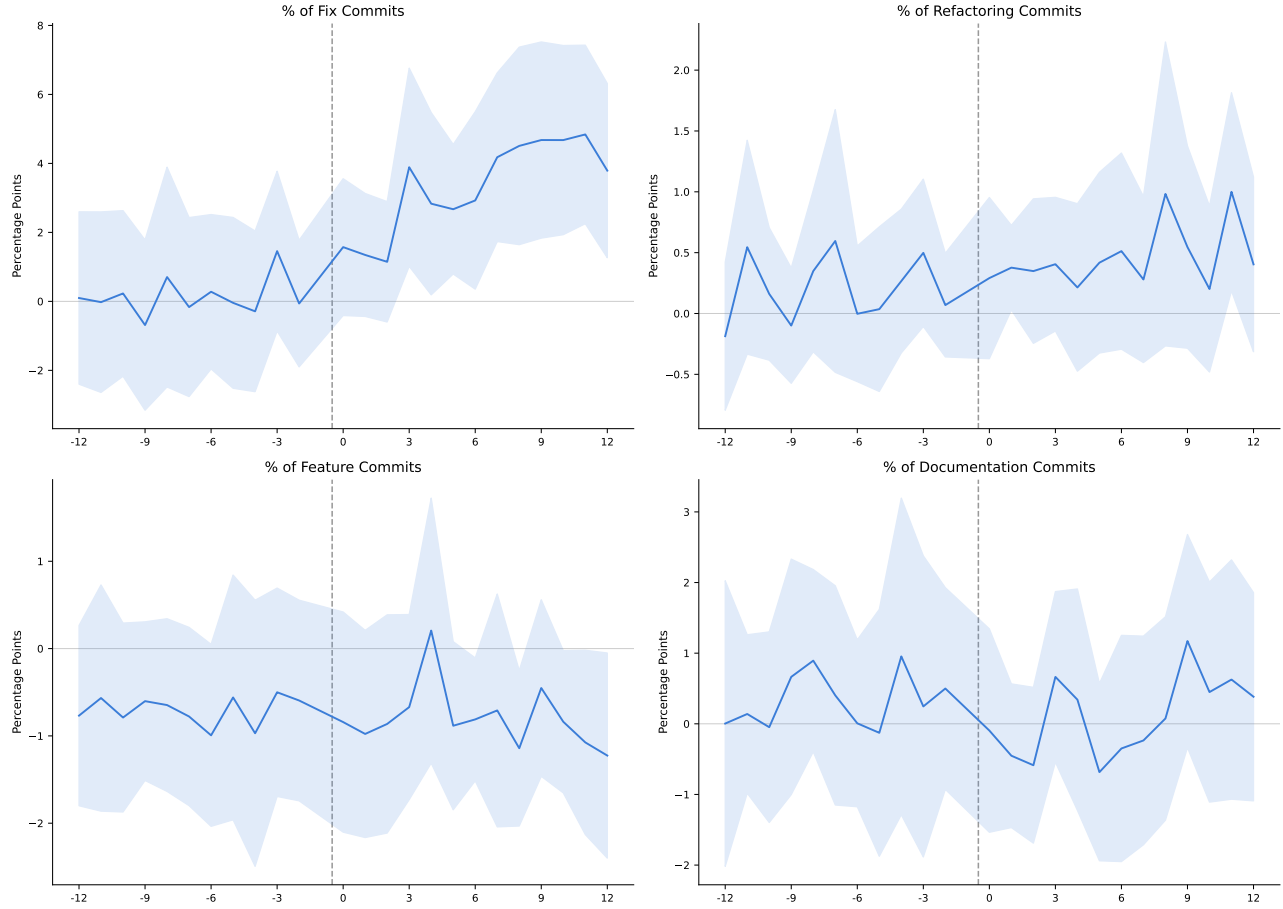


Figure 8: Cybersecurity Vulnerabilities and Development Composition (± 12 Months)

The top-left panel confirms the persistent rise in fix-related language documented in Table 15. The top-right panel examines refactoring keywords (“refactor,” “cleanup,” “clean-up,” “rewrite,” “rework”), which capture maintenance effort beyond immediate bug fixes. The dynamic coefficients are directionally positive throughout the post-treatment period, with individually significant estimates emerging at longer horizons—consistent with refactoring being a deliberate architectural remediation that firms undertake later.

The bottom-left panel displays a pattern consistent with the resource-diversion channel: feature-related keywords (“feature,” “implement,” “enhancement,” “introduce,” “functionality”) exhibit a progressive negative trend post-shock, with individually significant coefficients emerging at horizons $t = 6$ through $t = 12$, despite the static average being modest and not statistically significant ($\hat{\beta} = -0.15$ p.p., $p = 0.30$). This gradual decline is suggestive of crowding-out that materializes as teams remain absorbed in remediation and refactoring, leaving fewer engineering resources for developing new product capabilities. The bottom-right panel displays documentation keywords (“docs,” “readme,” “comment,” “documentation”) as a control category. Because documentation effort has no mechanical link to vulnerability remediation, a null result here supports the interpretation that the patterns in the

other panels reflect remediation-specific behavior rather than unrelated shifts in engineering practices. Consistent with this logic, the dynamic coefficients are close to zero and statistically insignificant throughout, reinforcing the specificity of the remediation channel.

3.4 Robustness

We subject our main findings to a rich set of robustness and placebo tests. In a stacked difference-in-differences design with staggered treatment timing, parallel trends are a maintained assumption, and Roth (2022) demonstrates that testing the significance of the pre-treatment coefficients may have limited power to detect violations of parallel pre-trends. We therefore report joint Wald tests of pre-treatment coefficients for all five firm-level outcomes and complement these with placebo and stability exercises that do not rely solely on pre-period evidence.

For four of our five outcomes, joint tests of pre-treatment coefficients cannot reject the null of parallel trends at conventional levels: technical debt ($p = 0.44$), cognitive complexity ($p = 0.12$), acquisition probability ($p = 0.20$), and funding ($p = 0.11$). For headcount, technical debt, and cognitive complexity, the test uses all eleven monthly pre-treatment interaction coefficients; acquisition and funding use three quarterly coefficients, matching their respective event-study specifications. For headcount, the joint test rejects the null ($F(11) = 30.1$, $p = 0.002$). We note, however, that the pre-period point estimates for headcount are slightly positive, indicating that treated firms were, if anything, growing marginally faster than controls prior to vulnerability discovery. This direction of pre-treatment divergence works against our finding of a negative post-treatment effect: if the pre-existing trend continued into the post-period, it would bias our estimate toward zero, making our finding of slower post-treatment headcount growth conservative. We acknowledge this imperfect pre-trend and address it through the additional robustness exercises described below.

To assess whether any single vulnerability event drives our results, we conduct a leave-one-cohort-out analysis, re-estimating the baseline specification 38 times, each time dropping one cohort from the stacked sample. Figures 9 and 10 display the estimated treatment effect from each iteration alongside the full-sample estimate. For headcount, the coefficient ranges from -0.091 to -0.066 , tightly bracketing the full-sample estimate of -0.081 . For technical debt, the range is $[0.169, 0.223]$ relative to the full-sample coefficient of 0.204 . No single cohort is pivotal for either outcome. This stability is noteworthy in stacked designs, where an outlier cohort could, in principle, dominate the pooled estimate.

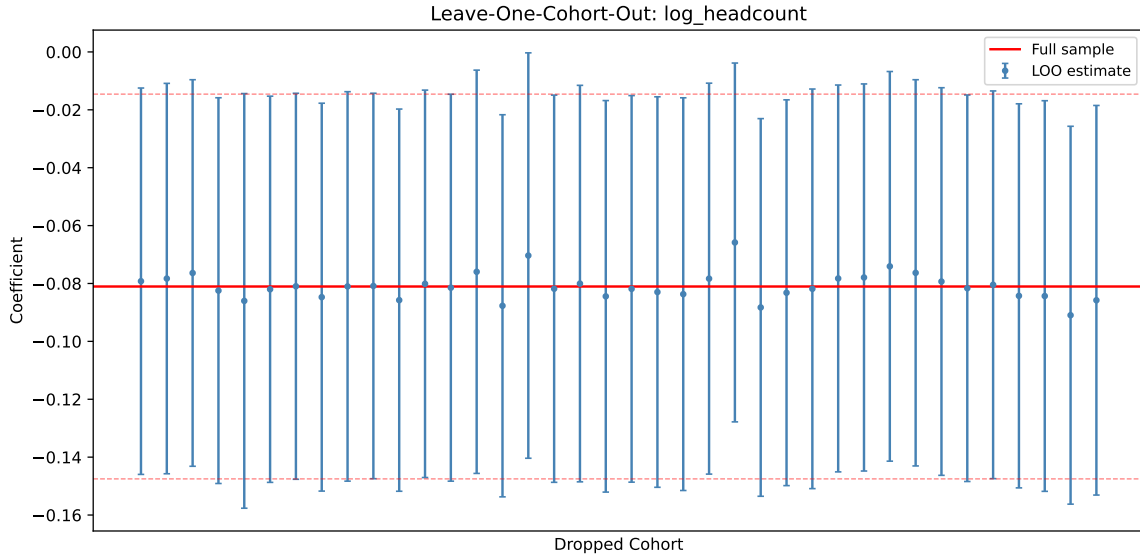


Figure 9: Leave-One-Cohort-Out Estimates: Ln(Headcount)

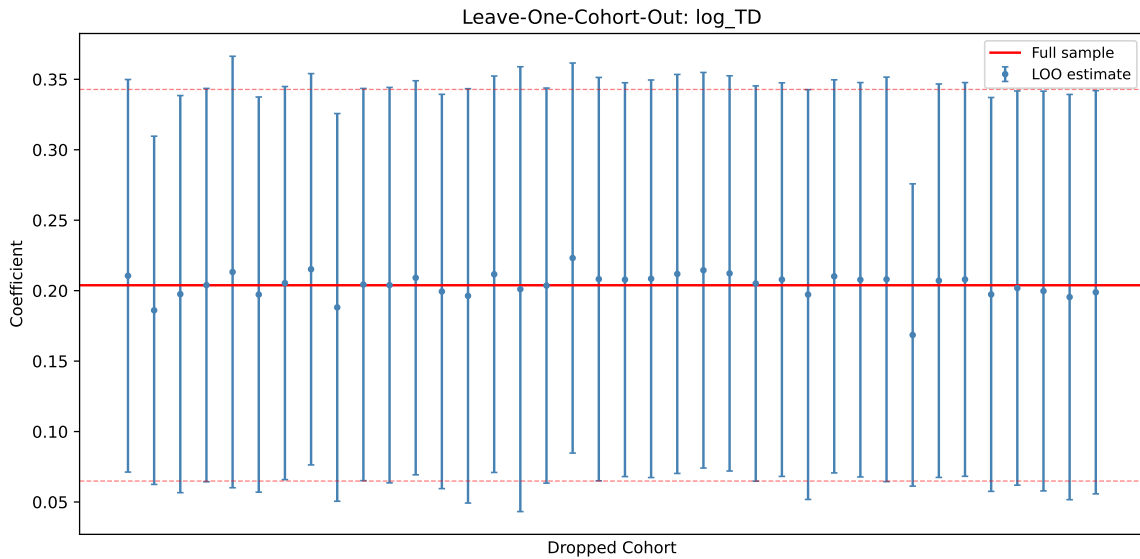


Figure 10: Leave-One-Cohort-Out Estimates: Ln(Technical Debt)

To assess the statistical significance of our main estimates without relying on asymptotic distributional assumptions, we conduct randomization inference (RI) by randomly permuting treatment assignment across cohorts 1,000 times and re-estimating the baseline specification for each permutation. Figures 11 and 12 display the distribution of placebo treatment effects alongside the observed estimate (vertical line). For technical debt, the observed coefficient exceeds all but 9 of the 1,000 placebo estimates (RI $p = 0.009$). For headcount, the observed estimate exceeds roughly 92% of place-

bos (RI $p = 0.076$), consistent with the asymptotic inference. These RI results provide nonparametric evidence against the null hypothesis that treatment effects arise from chance assignment.

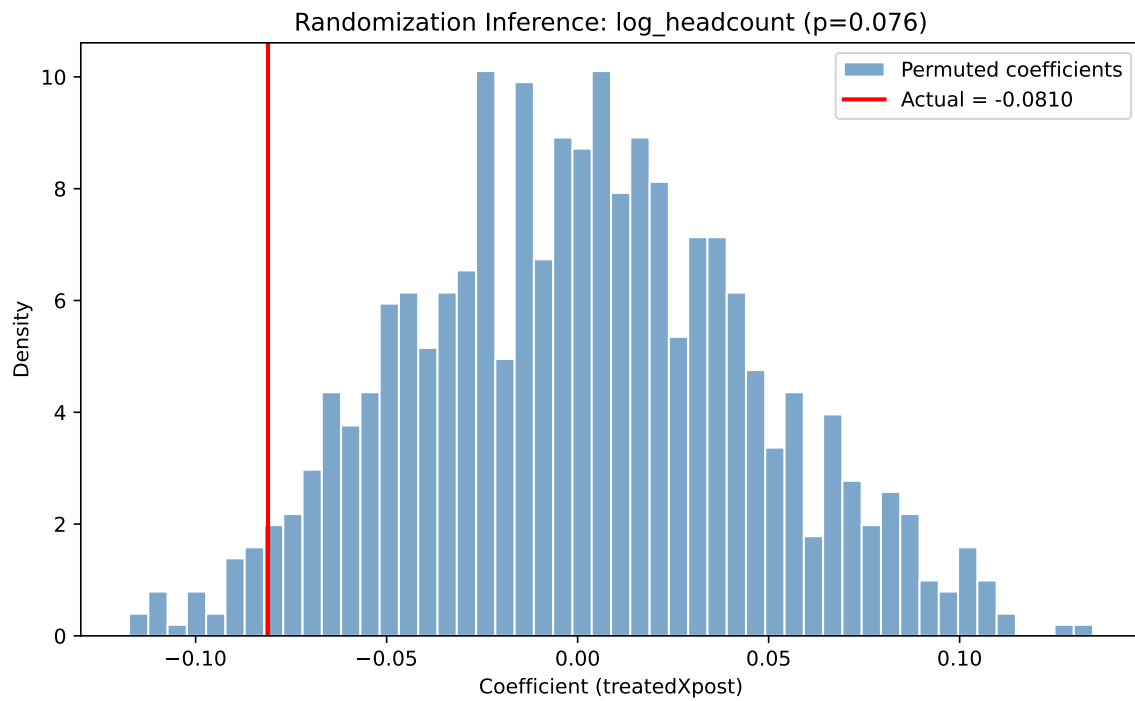


Figure 11: Randomization Inference: Ln(Headcount)

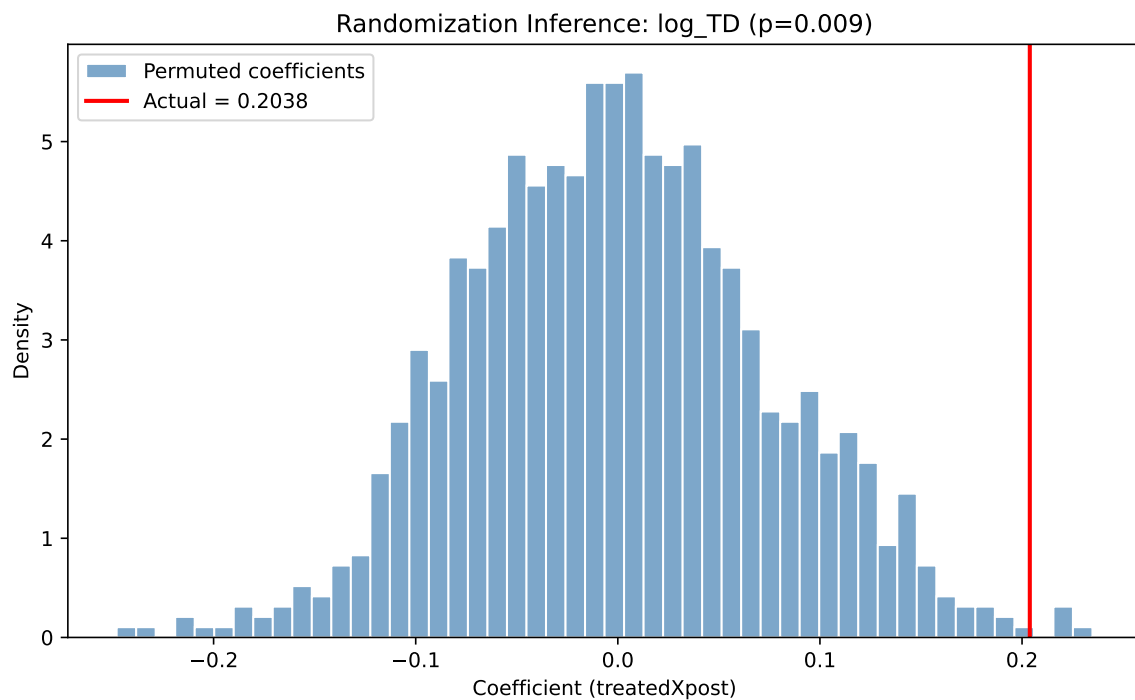


Figure 12: Randomization Inference: Ln(Technical Debt)

As shown in the development composition analysis (Figure 8), the long-term commit-level response is specific to fix-related remediation activity. Documentation keywords show precise null effects throughout the event window, ruling out the possibility that vulnerability discoveries trigger a general disruption to development workflows rather than targeted remediation.

Finally, we examine the sensitivity of our main results to the choice of event window. Both headcount and technical debt effects are statistically significant at the ± 6 , ± 9 , and ± 12 month windows. At the ± 18 month window, the technical debt estimate remains significant ($\hat{\beta} = 0.232$, $p = 0.016$) while the headcount estimate attenuates and loses significance ($p = 0.162$). This differential persistence is consistent with the nature of the two outcomes: headcount growth is a flow measure that captures a transitory adjustment in the pace of hiring, while technical debt is a stock variable that accumulates as suboptimal code persists in the codebase. As shown in column (4) of each table, all main results are robust to replacing language fixed effects with firm-type-by-calendar-month fixed effects, which absorb industry-segment-specific time trends (e.g., all security firms experiencing a common shock in a given month). The full suite of additional robustness tests—sensitivity to standard error clustering, dose-response heterogeneity by update severity and upgrade complexity, and window sensitivity—is reported in Appendix A.6.

4 Conclusion

This paper uses high-frequency codebase data to study how cybersecurity vulnerabilities affect firm outcomes and through what operational channels these effects persist. We construct a panel of 551 open-core technology firms spanning 2010–2025, analyze over 84,000 monthly code snapshots through static analysis, and exploit the exogenous discovery of critical-severity vulnerabilities in third-party software dependencies as shocks to firm performance. Because discovery timing is unpredictable and exposure depends on pre-existing dependency choices rather than firm quality, treatment is plausibly orthogonal to the unobservable characteristics that typically confound cybersecurity research. Our stacked difference-in-differences design—built on 88 unique vulnerability events, 38 cohorts, and 284 firms—provides the first causal evidence that cybersecurity vulnerabilities, independent of actual exploitation, impose economically meaningful costs on affected firms.

Treated firms experience approximately 8 percentage points lower headcount growth relative to clean controls—7.8% lower levels, eliminating roughly one-fifth of annual growth—and are 1.5–1.7 percentage points less likely to be acquired. We find no statistically significant effects on funding or short-run survival, consistent with shocks that reallocate resources and delay scaling rather than immediately threatening firm continuation. On the operational side, technical debt grows 16–20 percentage points faster in treated firms (17–23% higher levels), representing approximately \$30,700 in additional remediation liability—roughly 3.5 engineer-months of repair work—and cognitive complexity rises 16–21 percentage points faster (17–23% higher levels), both relative to controls in the

year following discovery. Panel regressions linking cognitive complexity to subsequent performance indicate that a 10% increase in complexity is associated with a 1.18–1.32 percentage point reduction in two-year-ahead headcount growth—an association that holds after controlling for new security issues and security hotspots. While this evidence is correlational, it is consistent with accumulated code liabilities constraining future firm performance.

The commit-level evidence traces the operational mechanism through which these effects arise. In the days immediately surrounding discovery, treated firms’ codebases receive approximately 2.7 additional commits per day relative to controls—an urgent remediation spike that attenuates sharply in wider windows. Vulnerability-resolution language (e.g., “upgrade,” “CVE,” “security-patch”) rises by 1.1–1.6 percentage points in windows up to ± 14 days but returns to baseline by ± 30 days, confirming a short-lived security-specific response. The broader consequences, however, persist: fix-related language increases by 3.2 percentage points at the monthly frequency and remains elevated for the following twelve months, indicating sustained remediation that extends well beyond the initial vulnerability-resolution spike. To examine whether the composition of development activity shifts from innovation toward maintenance, we classify commits into four categories—fix, refactoring, feature, and documentation—and estimate separate event studies for the share of commits in each category. Fix and refactoring language rises persistently in treated firms, while the share of feature-related commits exhibits a progressive decline at longer horizons ($t = +6$ through $t = +12$), consistent with crowding-out of new product development as engineering capacity remains absorbed in remediation. The coefficient on the share of documentation commits—a category with no mechanical link to vulnerability remediation—are close to zero and statistically insignificant throughout the event window, reinforcing that we are .

Taken together, the evidence is consistent with an emergency remediation channel through which transient vulnerability discoveries translate into persistent software liabilities. The initial response produces architecturally expedient patches under time pressure, which generate cascading downstream repair work that persists for months—manifesting as accumulated technical debt and cognitive complexity in the codebase. The progressive shift in the composition of development activity away from feature creation and toward maintenance is suggestive of crowding-out, whereby engineering resources are diverted from innovation to remediation in ways that compound over time.

Although our sample consists of small, private, predominantly venture-backed firms for which the per-firm magnitudes likely represent an upper bound, the economic stakes extend well beyond our sample through the software supply chain. The firms we study produce foundational infrastructure—databases, security libraries, developer platforms—that is deeply embedded in the technology stacks of large enterprises, financial institutions, and government agencies. Industry audits document that 96% of commercial codebases contain open-source components ([Synopsys, 2024](#)), and [Hoffmann et al. \(2024\)](#) estimate the demand-side value of widely used open-source software at \$8.8 trillion. A vulnerability in a widely adopted open-source dependency does not merely affect its developer; it

propagates downstream to every organization that depends on it. The core mechanism we document—exogenous dependency shocks triggering time-sensitive remediation that degrades code quality—does not depend on firm size or the open-core business model; it applies to any organization maintaining a software codebase that relies on third-party libraries.

These findings advance the literature along three dimensions. First, while prior work has documented market reactions to cyberattacks and traced the propagation of realized breaches through supply chains, we provide the first causal evidence that cybersecurity vulnerabilities—independent of actual exploitation—affect real firm outcomes, including headcount growth and acquisition probability. Second, we introduce technical debt as a measurable, dollar-denominated off-balance-sheet liability for technology firms, offering a framework that connects code quality to the finance literature on intangible assets, M&A due diligence, and software asset valuation. Third, we document an operational channel—emergency remediation generating persistent technical debt and cognitive complexity—through which transient security shocks translate into lasting constraints on firm performance, tracing it directly through the content and timing of individual code changes.

References

- Eli Amir, Shai Levi, and Gilad Livne. Do firms underreport information on cyber-attacks? evidence from capital markets. *Review of Accounting Studies*, 23(3):1177–1206, 2018. doi: 10.1007/s11142-018-9452-4. URL <https://doi.org/10.1007/s11142-018-9452-4>.
- Andrew C. Baker, David F. Larcker, and Charles C.Y. Wang. How much should we trust staggered difference-in-differences estimates? *Journal of Financial Economics*, 144(2):370–395, 2022. ISSN 0304-405X. doi: <https://doi.org/10.1016/j.jfineco.2022.01.004>. URL <https://www.sciencedirect.com/science/article/pii/S0304405X22000204>.
- Brantly Callaway and Pedro H.C. Sant’Anna. Difference-in-differences with multiple time periods. *Journal of Econometrics*, 225(2):200–230, 2021. ISSN 0304-4076. doi: <https://doi.org/10.1016/j.jeconom.2020.12.001>. URL <https://www.sciencedirect.com/science/article/pii/S0304407620303948>. Themed Issue: Treatment Effect 1.
- G. Ann Campbell. Cognitive complexity: A new way of measuring understandability. Technical report, SonarSource, August 2023. URL <https://www.sonarsource.com/docs/CognitiveComplexity.pdf>. Version 1.7.
- Annamaria Conti, Vansh Gupta, Jorge Guzman, and Maria P. Roche. Better keep the twenty dollars: Incentivizing innovation in open source. Working Paper 24-014, Harvard Business School, September 2023. Revised January 2025. Also available as NBER Working Paper Series, No. 31668, September 2023.
- Annamaria Conti, Christian Peukert, and Maria Roche. Beefing it up for your investor? engagement with open source communities, innovation, and startup funding: Evidence from github. *Organization Science*, 36(4):1551–1573, 2025. doi: 10.1287/orsc.2023.18348. URL <https://doi.org/10.1287/orsc.2023.18348>.
- Matteo Crosignani, Marco Macchiavelli, and AndréF. Silva. Pirates without borders: The propagation of cyberattacks through firms’ supply chains. *Journal of Financial Economics*, 147(2):432–448, 2023. ISSN 0304-405X. doi: <https://doi.org/10.1016/j.jfineco.2022.12.002>. URL <https://www.sciencedirect.com/science/article/pii/S0304405X2200246X>.
- Ward Cunningham. The WyCash portfolio management system. In *Addendum to the Proceedings on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA’92)*, pages 29–30. ACM, 1992. doi: 10.1145/157709.157715.
- Bill Curtis, Jay Sappidi, and Alexandra Szykarski. Estimating the principal of an application’s technical debt. *Software, IEEE*, 29:34–42, 11 2012. doi: 10.1109/MS.2012.156.

- João Dimas and Lampros Pnevmatikos. Venture capital, engineering quality, and technical debt. *SSRN Electronic Journal*, January 2026. doi: 10.2139/ssrn.6023094. URL <https://ssrn.com/abstract=6023094>.
- Andrea L. Eisfeldt and Dimitris Papanikolaou. Organization capital and the cross-section of expected returns. *The Journal of Finance*, 68(4):1365–1406, 2013. doi: 10.1111/jofi.12034.
- Michael Ewens, Ryan H. Peters, and Sean Wang. Measuring intangible capital with market prices. *Management Science*, 71(1):407–427, 2025. doi: 10.1287/mnsc.2021.02058.
- Chaim Fershtman and Neil Gandal. Direct and indirect knowledge spillovers: the “social network” of open-source projects. *The RAND Journal of Economics*, 42(1):70–91, 2011. doi: <https://doi.org/10.1111/j.1756-2171.2010.00126.x>. URL <https://onlinelibrary.wiley.com/doi/abs/10.1111/j.1756-2171.2010.00126.x>.
- Chris Florackis, Christodoulos Louca, Roni Michaely, and Michael Weber. Cybersecurity risk. *The Review of Financial Studies*, 36(1):351–407, 2023. ISSN 0893-9454. doi: 10.1093/rfs/hhac024. URL <https://doi.org/10.1093/rfs/hhac024>.
- Todd A. Gormley and David A. Matsa. Growing out of trouble? corporate responses to liability risk. *The Review of Financial Studies*, 24(8):2781–2821, 04 2011. ISSN 0893-9454. doi: 10.1093/rfs/hhr011. URL <https://doi.org/10.1093/rfs/hhr011>.
- Todd A. Gormley and David A. Matsa. Playing it safe? managerial preferences, risk, and agency conflicts. *Journal of Financial Economics*, 122(3):431–455, 2016. ISSN 0304-405X. doi: <https://doi.org/10.1016/j.jfineco.2016.08.002>. URL <https://www.sciencedirect.com/science/article/pii/S0304405X1630143X>.
- Manuel Hoffmann, Frank Nagle, and Yanuo Zhou. The value of open source software. Strategy Unit Working Paper 24-038, Harvard Business School, 2024.
- Guido W. Imbens and Donald B. Rubin. *Causal Inference for Statistics, Social, and Biomedical Sciences: An Introduction*. Cambridge University Press, 2015. doi: 10.1017/CBO9781139025751.
- Herb Krasner. Cost of poor software quality in the U.S. : A 2022 report. Technical report, Consortium for Information & Software Quality (CISQ), 2022. Accessed: March 25, 2025.
- Philippe Kruchten, Robert L. Nord, and Ipek Ozkaya. Technical debt: From metaphor to theory and practice. *IEEE Software*, 29(6):18–21, 2012. doi: 10.1109/MS.2012.167.
- Jean-Louis Letouzey. The SQALE method for evaluating technical debt. In *2012 Third International Workshop on Managing Technical Debt (MTD)*, pages 31–36, Zurich, Switzerland, June 2012. doi: 10.1109/MTD.2012.6225997. URL <https://ieeexplore.ieee.org/document/6225997>.

- Zengyang Li, Paris Avgeriou, and Peng Liang. A systematic mapping study on technical debt and its management. *Journal of Systems and Software*, 101:193–220, 2015. doi: 10.1016/j.jss.2014.12.027.
- Thomas J. McCabe. A complexity measure. *IEEE Transactions on Software Engineering*, SE-2(4): 308–320, December 1976. doi: 10.1109/TSE.1976.233837.
- McKinsey and Co. Tech debt: Reclaiming tech equity. *McKinsey & Company*, 2020. Accessed: March 25, 2025.
- Frank Nagle. Open source software and firm productivity. *Management Science*, 65(3):1191–1215, 2019. doi: 10.1287/mnsc.2017.2977. URL <https://doi.org/10.1287/mnsc.2017.2977>.
- Jonathan Roth. Pretest with caution: Event-study estimates after testing for parallel trends. *American Economic Review: Insights*, 4(3):305–322, 2022. doi: 10.1257/aeri.20210236.
- D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, and Michael Young. Machine learning: The high interest credit card of technical debt. In *SE4ML: Software Engineering for Machine Learning (NIPS 2014 Workshop)*, 2014.
- Inc. Stripe. The developer coefficient. <https://stripe.com/newsroom/stories/developer-coefficient>, 2018. Accessed: March 25, 2025.
- Liyang Sun and Sarah Abraham. Estimating dynamic treatment effects in event studies with heterogeneous treatment effects. *Journal of Econometrics*, 225(2):175–199, 2021. ISSN 0304-4076. doi: <https://doi.org/10.1016/j.jeconom.2020.09.006>. URL <https://www.sciencedirect.com/science/article/pii/S030440762030378X>. Themed Issue: Treatment Effect 1.
- Synopsys. 2024 open source security and risk analysis report. Technical report, Synopsys, Inc., 2024. Based on audit of 1,067 commercial codebases by Black Duck Audit Services.
- U.S. Bureau of Labor Statistics. Software developers, quality assurance analysts, and testers. <https://www.bls.gov/ooh/computer-and-information-technology/software-developers.htm>, 2025. Median pay: \$63.20 per hour (May 2024 data).

A Appendix

Table 16: Variable Definitions

Variable	Definition
<i>A. Firm outcomes and characteristics</i>	
Headcount	Total number of employees at firm i in period t , constructed from LinkedIn position histories. Our main growth outcome uses $\ln(\text{Headcount})$.
Software engineer (SWE) headcount	Number of employees in software engineering roles at firm i in period t , based on a job-title mapping applied to LinkedIn position histories.
Funding round	Indicator equal to 1 if firm i receives a financing event in period t (Seed Round, Early Stage VC, Late Stage VC, PE Growth/Expansion, Accelerator/Incubator, or Angel investment), based on PitchBook deal records.
Acquisition	Indicator equal to 1 in and after the first observed Merger/Acquisition or Buyout/LBO event for firm i , based on PitchBook deal records.
Is Alive	Indicator equal to 1 until the first period in which PitchBook lists firm i as out of business; 0 thereafter.
Founding year	Year the organization was founded (PitchBook).
Product type	Firm product-category classification (10 categories; see Table 1) constructed via large language model consensus using firm descriptions. These are grouped into five broader firm types for Column (4) fixed effects.
$Growth_{i,t+24}$	Headcount growth over the subsequent 24 months: $\text{Headcount}_{i,t+24}/\text{Headcount}_{i,t} - 1$.
<i>B. Cybersecurity shocks and stacked DiD variables</i>	
Critical dependency vulnerability (event)	Public discovery of a critical-severity vulnerability (CVSS score ≥ 9) affecting a third-party software dependency used by a firm, sourced from the GitHub Advisory Database (GitHub-reviewed subset; merged with NVD metadata). The event month is the discovery month.

Continued on next page

Table 16 – continued from previous page

Variable	Definition
Common Vulnerability Scoring System (CVSS)	Standardized 0–10 severity scale used to quantify the severity and exploitability of software vulnerabilities. We define critical discoveries as those with CVSS ≥ 9 .
Treated	Indicator equal to 1 if firm i is exposed to the cohort’s critical dependency vulnerability; treated firms are first exposed at $t = 0$ within each stacked cohort.
Post	Indicator equal to 1 for periods strictly after the cohort event month ($t \geq 1$) and 0 otherwise; the discovery month ($t = 0$) is coded as pre-period since treatment exposure is partial during that month.
Treated $\times$ Post ($did_{i,c,t}$)	Main treatment indicator in the stacked difference-in-differences specification, equal to $Treated_{i,c} \times Post_{c,t}$.
Cohort (c)	Event-specific cohort defined by a unique (dependency, discovery month) pair; cohorts are stacked to form the analysis sample.
Relative time (t)	Event time measured relative to each cohort’s discovery month (reported at either monthly or quarterly frequency depending on the specification).
Cohort $\times$ firm fixed effects ($\chi_{c,i}$)	Firm-by-cohort fixed effects that absorb time-invariant firm characteristics within each stacked cohort.
Time $\times$ cohort fixed effects ($\delta_{c,t}$)	Time-by-cohort fixed effects that absorb common shocks affecting all firms within a cohort at event time t .
Upgrade severity (Major/Minor)	Classification of the required dependency update following semantic versioning conventions: major upgrades introduce breaking API changes, while minor upgrades are backward-compatible. Used to stratify treatment effects in dose-response heterogeneity tests.
<i>C. Codebase scale and engineering-quality metrics (SonarQube)</i>	
Lines of code (LoC)	Total number of non-comment lines of source code in the firm’s product repository(ies) at time t (SonarQube NCLOC). When aggregating to the firm level, we sum across all repositories observed for the firm in period t . Logged versions use $\ln(\text{LoC} + 1)$.

Continued on next page

Table 16 – continued from previous page

Variable	Definition
Open issues	Stock of unresolved SonarQube issues (reliability, security, or maintainability) currently active in the codebase at time t . Logged versions use $\ln(\text{Open Issues} + 1)$.
Development cost	SonarQube's estimated development cost for the codebase at time t ; used to scale technical debt into a share measure.
Critical violations	Stock of critical-severity code violations (reliability, security, or maintainability) flagged by SonarQube at time t . Logged versions use $\ln(\text{count} + 1)$.
Bugs per kLoC	Number of SonarQube-detected bug issues per thousand non-comment lines of code at time t .
Duplicated lines	Total number of duplicated lines of code across the firm's repository(ies) at time t , as reported by SonarQube.
Dominant programming language	The most common programming language across the firm's repositories in period t , used to construct language-by-cohort and language-by-month fixed effects.
<i>D. Security and technical debt metrics (SonarQube)</i>	
Technical debt (TD, USD)	Total dollar value of technical debt at time t , defined as the SQALE remediation time for maintainability issues converted into dollars using an assumed \$55 hourly software engineer wage. When aggregating to the firm level, we sum TD across all repositories observed for the firm in period t .
Technical Debt per Line of Code	Technical debt divided by lines of code (LoC): TD/LoC .
Technical Debt Velocity	Technical debt on new code per new line of code, constructed as $(\text{new technical debt} \times \$55)$ divided by new lines when new lines > 0 .

Continued on next page

Table 16 – continued from previous page

Variable	Definition
Cognitive Complexity	A SonarQube metric that quantifies how difficult code is to understand and maintain. Unlike cyclomatic complexity (which counts decision paths), cognitive complexity weights structures based on mental effort required: nested conditions, non-linear control flow, and recursive patterns receive higher scores. When aggregating to the firm level, we sum cognitive complexity across all repositories. Logged versions use $\ln(\text{Cognitive Complexity} + 1)$.
Security Hotspots	Security-sensitive code segments flagged by SonarQube as requiring manual audit to assess vulnerability. We use (i) new security hotspots (flow) at time t , (ii) hotspots scaled by dividing with their sample standard deviation.
New security issues (High)	Number of new high-severity security issues detected by SonarQube in the firm's repository(ies) during period t . Captures the flow of newly introduced security vulnerabilities. Logged versions use $\ln(\text{count} + 1)$.
Vulnerabilities per kLoC	Number of SonarQube-detected vulnerability issues per thousand non-comment lines of code at time t .
Security issues (High) per kLoC	Number of high-severity security issues per thousand non-comment lines of code at time t .
<i>E. Commit-level activity metrics</i>	
Commits	Total number of commits to the repository in period t . Logged versions use $\ln(\text{Commits} + 1)$.
Fix keywords per commit	Ratio of commits containing fix/patch-related keywords (e.g., fix, patch, hotfix, bugfix, resolve, repair, correct) to total commits in period t . Captures remediation-related development activity.
Average message length	Mean length (in characters) of commit messages in period t . Logged versions use $\ln(\text{Avg Message Length} + 1)$. Longer messages may indicate more detailed documentation of changes.
Days with commits	Number of days in period t with at least one commit. Controls for development intensity when analyzing per-commit metrics.

Continued on next page

Table 16 – continued from previous page

Variable	Definition
Feature keywords per commit	Proportion of commits containing feature-related keywords (e.g., feature, implement, enhancement) in period t . Used as a placebo outcome in keyword tests.
Documentation keywords per commit	Proportion of commits containing documentation-related keywords (e.g., docs, documentation, readme, changelog) in period t . Used as a placebo outcome in keyword tests.
Performance keywords per commit	Proportion of commits containing performance-related keywords (e.g., optimize, speed, cache, latency) in period t . Used as a placebo outcome in keyword tests.

Note: Codebase metrics are measured using SonarQube static analysis.

A.1 CVSS Scoring Framework

A.2 The Common Vulnerability Scoring System (CVSS)

The Common Vulnerability Scoring System (CVSS) is an open, industry-standard framework for communicating the characteristics and severity of software vulnerabilities. CVSS is owned and managed by FIRST.Org, Inc. (FIRST), a US-based non-profit organization whose mission is to help computer security incident response teams across the world. The system provides a standardized, vendor-agnostic methodology for quantifying vulnerability severity, enabling consistent assessment across different software products, vendors, and organizational environments.

We primarily use CVSS v3.1 Base scores in this study; when v3.1 is unavailable, we use the most recent available version (v3.0 or, rarely, v2.0). CVSS produces numerical scores ranging from 0 to 10, where higher scores indicate greater severity. These scores are derived from three metric groups: Base, Temporal, and Environmental. The Base metrics capture the intrinsic characteristics of a vulnerability that remain constant over time and across user environments, assuming a reasonable worst case impact across different deployed environments. The Temporal metrics reflect characteristics that change over time, such as the availability of exploit code or official fixes and workarounds. The Environmental metrics allow organizations to customize scores based on the importance of affected assets and the presence of mitigating controls in their specific computing environment.

The CVSS specification explicitly assumes that scoring occurs after the attacker has already located and identified the vulnerability; therefore, the difficulty of vulnerability discovery is not factored into the severity score.

Base Metrics and Scoring. The Base Score is the foundation of CVSS and reflects the severity of a vulnerability according to its intrinsic characteristics. It is composed of two sub-groups: Exploitability metrics and Impact metrics.

The *Exploitability metrics* capture the ease and technical means by which a vulnerability can be exploited:

- **Attack Vector (AV):** The context by which exploitation is possible (Network, Adjacent, Local, or Physical). Holding other metrics constant, Network attack vector yields the highest contribution to the Base score.
- **Attack Complexity (AC):** The conditions beyond the attacker's control that must exist to exploit the vulnerability (Low or High). All else equal, Low complexity vulnerabilities contribute more to the Base score.
- **Privileges Required (PR):** The level of privileges an attacker must possess before exploiting the vulnerability (None, Low, or High). All else equal, vulnerabilities requiring no privileges contribute most to the Base score.

- **User Interaction (UI):** Whether exploitation requires action from a user other than the attacker (None or Required). All else equal, vulnerabilities requiring no user interaction contribute more to the Base score.

The *Impact metrics* capture the direct consequences of successful exploitation on three dimensions:

- **Confidentiality (C):** The impact on the confidentiality of information resources (High, Low, or None). High indicates complete loss of confidentiality.
- **Integrity (I):** The impact on the integrity and trustworthiness of information (High, Low, or None). High indicates the attacker can modify any/all protected files.
- **Availability (A):** The impact on the availability of the affected component (High, Low, or None). High indicates complete denial of access to resources.

An additional metric, **Scope (S)**, captures whether a vulnerability in one component can affect resources beyond its security scope (Unchanged or Changed). A scope change occurs when the impact breaches a security boundary, affecting components managed by different security authorities.

The Base Score formula combines these metrics through a weighted calculation that produces a score from 0.0 to 10.0. Vulnerabilities with easier exploitation paths (e.g., network-accessible, low complexity, no privileges required) and higher impacts (e.g., complete loss of confidentiality, integrity, and availability) receive higher scores.

Qualitative Severity Ratings. CVSS maps numerical scores to qualitative severity ratings to facilitate communication and prioritization. Table 17 presents the standard mapping used throughout this study.

Table 17: CVSS v3.1 Qualitative Severity Rating Scale

Rating	CVSS Score Range
None	0.0
Low	0.1 – 3.9
Medium	4.0 – 6.9
High	7.0 – 8.9
Critical	9.0 – 10.0

Critical-Severity Vulnerabilities. Our analysis focuses exclusively on *critical-severity* vulnerabilities—those with CVSS Base Scores of 9.0 or above. A CVSS Base score does not indicate that exploitation occurred (or will occur), nor whether a specific organization was patched at the time of disclosure;

it is a standardized measure of potential severity under the assumption of a reasonable worst-case impact. We classify vulnerabilities as critical using the CVSS v3.x qualitative severity threshold (Base ≥ 9.0). When only CVSS v2.0 scores are available, we apply the same numeric cutoff as a high-severity screen; results are robust to excluding the small number of v2-only vulnerabilities in our sample. This restriction to critical-severity vulnerabilities serves multiple purposes for our research design. First, it narrows the range of severity, improving comparability across shocks in our treatment group. Second, vulnerabilities in this range are commonly treated as high priority in vulnerability management processes and likely to be prioritized for remediation due to their high potential severity. Third, by focusing on the most severe vulnerabilities, we study disclosure events more likely to prompt rapid triage and remediation activity and potentially divert engineering resources from other activities.

Many critical-severity vulnerabilities are characterized by high exploitability (such as network-based attack vectors with low complexity) combined with severe potential impacts on confidentiality, integrity, or availability. For example, a vulnerability allowing remote code execution without authentication that results in complete system compromise would typically receive a critical rating. However, critical scores can arise from different combinations of exploitability and impact metrics, particularly when scope changes occur. In our sample of third-party dependency vulnerabilities (see Table 19 in the Appendix), the mean CVSS Base score is 9.4, with all treated vulnerabilities scoring between 9.0 and 10.0 by construction.

Vector Strings. CVSS communicates vulnerability characteristics through vector strings—compact textual representations of all metric values. A vector string begins with “CVSS:3.1/” followed by abbreviated metric names and values separated by forward slashes. For example:

CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

This vector represents a vulnerability with Network attack vector, Low complexity, no Privileges Required, no User Interaction, Unchanged scope, and High impact on Confidentiality, Integrity, and Availability. This combination produces a Base Score of 9.8, placing it firmly in the critical-severity range. The vector string provides a transparent, reproducible representation of how a given severity score was derived, allowing readers to understand which characteristics drive the overall assessment.

Application in This Study. We obtain CVSS scores and associated metadata from two widely used sources: the GitHub Advisory Database and the National Vulnerability Database (NVD), operated by the U.S. National Institute of Standards and Technology (NIST). We rely on CVSS Base scores as reported in these databases. We prioritize version first (v3.1 > v3.0 > v2.0); within a given version, if multiple sources report scores, we use the GitHub Advisory Database score, as it reflects the GitHub security review process specific to the open-source ecosystem our firms operate in. Given that the vast majority of vulnerabilities in our sample period (2016–2025) have v3.0 or v3.1 scores, the number of v2-only scored vulnerabilities is negligible. Our focus on Base Scores (rather than Temporal or

Environmental scores) reflects the fact that Base metrics are publicly reported and consistent across all affected organizations, enabling a standardized measurement of shock severity. By restricting our analysis to critical-severity vulnerabilities affecting third-party dependencies, we study disclosure events with high standardized severity that are plausibly exogenous to individual firm characteristics.

A.3 Language-Specific Analysis Configurations

Appendix file missing: `air/appendix/language_specific_analysis_configurations.tex`

A.4 Commit Message Length Analysis

Table 18: Difference-in-Differences Estimates for Commit Message Length

	Ln(Avg Message Length)	
	(1)	(2)
Treated $\times$ Post	0.069*** (0.023)	0.068*** (0.023)
Days with Commits		0.003*** (0.001)
Repo $\times$ Cohort FE	Yes	Yes
Relative Month $\times$ Cohort FE	Yes	Yes
R^2	0.763	0.763
# Obs	326,733	326,733

Note: This table presents stacked difference-in-differences estimates of the effect of critical vulnerability discoveries on average commit message length. The dependent variable is the natural log of average commit message length (in characters) per repository-month. The sample consists of repository-month observations for treated repositories and clean control repositories. Standard errors in parentheses are clustered at the repository level. * $p < 0.1$, ** $p < 0.05$, *** $p < 0.01$.

A.5 Cognitive Complexity: Overview

Cognitive Complexity is a metric developed by SonarSource (and implemented in SonarQube) to measure the understandability of code (Campbell, 2023). Unlike Cyclomatic Complexity—formulated by McCabe (1976) to measure testability by counting linearly independent paths through a program—Cognitive Complexity is designed to quantify the mental effort required for a human to read and comprehend code. This distinction is economically meaningful: two methods with identical Cyclomatic Complexity scores can differ dramatically in how difficult they are to understand, modify, and maintain. The following provides a high-level summary; see Campbell (2023) for the complete specification.

Core Principles. Cognitive Complexity is assessed according to three basic rules:

1. **Ignore shorthand structures.** Features that condense multiple statements into more readable forms (e.g., null-coalescing operators) do not increment complexity. Similarly, method definitions themselves receive no baseline increment, unlike Cyclomatic Complexity. This rewards coding practices that improve readability.
2. **Increment for breaks in linear flow.** Structures that interrupt the natural top-to-bottom, left-to-right reading of code require additional mental effort. These include loops (`for`, `while`, `do-while`), conditionals (`if`, ternary operators), `switch` statements, `catch` blocks (but not `try` or `finally`), jumps to labels (`goto`, `break LABEL`, `continue LABEL`), sequences of binary logical operators, and recursion. Note that `else if` and `else` receive hybrid increments but do not incur nesting penalties, as the mental cost is already paid when reading the initial `if`.
3. **Increment for nesting.** When flow-breaking structures are nested inside one another, additional increments are assessed for each level of nesting. This captures the intuition that deeply nested code imposes compounding cognitive load.

Key Differences from Cyclomatic Complexity. Several design choices distinguish Cognitive Complexity from its predecessor:

- **Switch statements:** Cyclomatic Complexity treats each case as a separate branch, assigning one increment per case. Cognitive Complexity assigns a single increment for the entire `switch`, reflecting that comparing a single variable to explicit literal values is cognitively simpler than parsing an `if-else if` chain with arbitrary conditions.
- **Logical operators:** Rather than incrementing for each binary logical operator, Cognitive Complexity increments once per *sequence* of like operators. The expression `a && b && c && d` receives one increment, while `a || b && c || d` receives three (one per sequence of like operators), reflecting that mixed operators require more careful parsing.

- **Nesting penalties:** Cyclomatic Complexity does not account for nesting depth. Cognitive Complexity applies cumulative nesting increments, so a conditional nested inside a loop inside another conditional incurs progressively higher penalties.
- **Modern constructs:** Cognitive Complexity handles `try/catch` blocks (incrementing for `catch` but not `try` or `finally`), lambdas (which increment nesting level but not structural complexity), and recursion (which receives a fundamental increment per method in the recursion cycle).
- **Aggregation:** Because Cognitive Complexity does not assign a baseline increment to each method (unlike Cyclomatic Complexity’s minimum score of one), aggregate scores at the class or application level are meaningful. A domain class with many simple getters and setters will have low aggregate complexity, while a class with complex control flow will score high—regardless of method count.

Interpretation. SonarQube reports aggregate Cognitive Complexity at the repository level (summing across all methods in the codebase). In our analysis, we sum this repository-level measure across all repositories a firm commercializes to obtain firm-level Cognitive Complexity, then apply a $\ln(x + 1)$ transformation for regression analysis. Higher scores indicate codebases that are more difficult for developers to understand, modify, and extend. When teams face time pressure—such as during vulnerability remediation—they may write code that solves the immediate problem but introduces nested conditionals, complex error handling, or convoluted control flow. This accumulated complexity represents a form of “cognitive debt” that imposes ongoing costs on future development.

Table 19: Vulnerability Severity Distribution (GitHub Advisory Database)

Severity	Count	Frequency (%)
MODERATE	10,485	44.12
HIGH	8,317	35.00
CRITICAL	3,416	14.37
LOW	1,547	6.51

Note: Counts refer to the GitHub-reviewed subset of the GitHub Advisory Database across all ecosystems. Our causal shock sample restricts to CRITICAL severity (CVSS ≥ 9.0) and matched firm exposures. The observed severity distribution likely reflects selection in reporting: security researchers and maintainers have stronger incentives to report and document high-severity vulnerabilities, so lower-severity issues are likely underrepresented.

Table 20: Vulnerability Distribution by Software Ecosystem (GitHub Advisory Database)

Ecosystem	Count	Frequency (%)	Cumulative Frequency (%)
Maven	5,764	24.25	24.25
Packagist	4,788	20.15	44.40
npm	4,117	17.32	61.73
PyPI	3,883	16.34	78.06
Go	2,501	10.52	88.59
crates.io	1,018	4.28	92.87
RubyGems	893	3.76	96.63
NuGet	681	2.87	99.50
SwiftURL	37	0.16	99.65
GitHub Actions	36	0.15	99.80
Hex	36	0.15	99.95
Pub	11	0.05	100.00

Notes: Counts refer to the GitHub-reviewed subset of the GitHub Advisory Database. Each ecosystem corresponds to a language-specific package management system: *Maven* (Java/JVM), *Packagist* (PHP), *npm* (JavaScript/TypeScript), *PyPI* (Python), *Go* (Go modules), *crates.io* (Rust), *RubyGems* (Ruby), *NuGet* (.NET including C# and F#), *SwiftURL* (Swift Package Manager), *GitHub Actions* (reusable CI/CD workflows), *Hex* (Elixir and Erlang), and *Pub* (Dart/Flutter).

Table 21: Vulnerability Discoveries by Year (GitHub Advisory Database)

Year	Count	Frequency (%)
2017	221	0.93
2018	785	3.30
2019	745	3.13
2020	1,398	5.88
2021	2,610	10.98
2022	8,523	35.86
2023	3,332	14.02
2024	3,654	15.38
2025	2,497	10.51

Note: Counts refer to the GitHub-reviewed subset of the GitHub Advisory Database across all severity levels.

A.6 Additional Robustness Tests

This appendix reports additional robustness and placebo tests supplementing Section 3.4. We present pre-trend diagnostics, window sensitivity, clustering sensitivity, and dose-response heterogeneity. Leave-one-cohort-out analysis, randomization inference, and keyword placebo tests are reported in the main text.

Pre-Trend F-Tests. Table 22 reports joint Wald tests of the null hypothesis that all pre-treatment dynamic coefficients are jointly equal to zero. For headcount, technical debt, and cognitive complexity, the test uses all eleven monthly pre-treatment interaction coefficients; acquisition and funding use three quarterly coefficients, matching their respective event-study specifications. For four of five firm-level outcomes, the test cannot reject the null at conventional levels: technical debt ($p = 0.44$), cognitive complexity ($p = 0.12$), acquisition probability ($p = 0.20$), and funding ($p = 0.11$). For headcount, the joint test rejects ($F(11) = 30.1$, $p = 0.002$). As discussed in Section 3.4, the pre-period point estimates for headcount are slightly positive, indicating that treated firms were growing marginally faster prior to vulnerability discovery. This direction of pre-treatment divergence biases against our finding of a negative post-treatment effect, making our headcount estimate conservative (Roth, 2022).

Table 22: Joint Wald Tests of Pre-Treatment Dynamic Coefficients

Outcome	Pre-Period Coefficients	Test Statistic	p -Value
Ln(Headcount)	11	30.134	0.002
Ln(Technical Debt)	11	10.988	0.444
Ln(Cognitive Complexity)	11	16.489	0.124
Acquisition	3	4.701	0.195
Funding	3	5.943	0.114

Note: This table reports joint Wald tests of the null hypothesis that all pre-treatment dynamic coefficients are jointly equal to zero. For headcount, technical debt, and cognitive complexity, the test uses all eleven monthly pre-treatment interaction coefficients (months -12 through -2 , with month -1 as the reference). For acquisition and funding, the test uses three quarterly pre-treatment coefficients, matching those outcomes' event-study specifications. $p > 0.05$ indicates failure to reject the null of parallel pre-trends. Standard errors are clustered at the firm level.

Window Sensitivity. Table 23 re-estimates the baseline firm-level specification across four event windows (± 6 , ± 9 , ± 12 , and ± 18 months). Our main results use the ± 12 month window. Technical debt is robust across all windows, significant at the 5% level or better in every specification. Headcount is significant at the ± 6 , ± 9 , and ± 12 month windows but attenuates at ± 18 months ($p = 0.162$). This differential persistence is consistent with the nature of the two outcomes: headcount growth is a flow measure that captures a transitory adjustment in the pace of hiring, while technical debt is a stock variable that accumulates as suboptimal code persists in the codebase. The larger coefficient magnitudes at shorter windows for headcount and at longer windows for technical debt further support this flow-versus-stock interpretation.

Table 23: Sensitivity of Main Results to Event Window Length

Window	Outcome	Coefficient	SE	p -Value	Obs
± 6 months	Ln(Headcount)	-0.067^{***}	(0.024)	0.008	70,943
± 6 months	Ln(Technical Debt)	0.129^{**}	(0.052)	0.017	70,943
± 9 months	Ln(Headcount)	-0.085^{***}	(0.029)	0.006	97,046
± 9 months	Ln(Technical Debt)	0.163^{***}	(0.058)	0.008	97,046
± 12 months	Ln(Headcount)	-0.081^{**}	(0.034)	0.022	114,393
± 12 months	Ln(Technical Debt)	0.204^{***}	(0.071)	0.007	114,393
± 18 months	Ln(Headcount)	-0.066	(0.046)	0.162	143,933
± 18 months	Ln(Technical Debt)	0.232^{**}	(0.091)	0.016	143,933

Note: This table examines the sensitivity of the main firm-level results to the choice of event window. Each row re-estimates the Column (1) baseline stacked DiD specification (firm-by-cohort and event-time-by-cohort fixed effects, no additional controls) with a different symmetric window around the vulnerability discovery date. The main results use ± 12 months. Controls that have not experienced any critical dependency vulnerability during the respective event window serve as the counterfactual. The number of cohorts varies across windows because some vulnerability events fall too close to the sample boundary to accommodate wider windows. Standard errors in parentheses are clustered at the firm level. * $p < 0.1$, ** $p < 0.05$, *** $p < 0.01$.

Clustering Sensitivity. Our main specifications cluster standard errors at the firm level. Table 24 reports the baseline specifications under three alternative clustering structures: firm-only (our main approach), two-way clustering at the firm and cohort level, and cohort-only clustering. The point estimate is mechanically identical across structures (only standard errors change). Both outcomes are significant at the 10% level or better under all three clustering approaches, and technical debt is significant at the 5% level or better in every case. Firm-level clustering is the most conservative of the three approaches, producing the largest standard errors; our choice of this structure therefore ensures that reported significance levels are not an artifact of the variance estimation approach.⁶

Table 24: Sensitivity of Main Results to Clustering Structure

Outcome	Clustering	Coefficient	SE	<i>p</i> -Value
Ln(Headcount)	Firm only	−0.081*	(0.042)	0.057
Ln(Headcount)	Cohort + Firm	−0.081**	(0.034)	0.022
Ln(Headcount)	Cohort only	−0.081***	(0.028)	0.006
Ln(Technical Debt)	Firm only	0.204**	(0.095)	0.033
Ln(Technical Debt)	Cohort + Firm	0.204***	(0.071)	0.007
Ln(Technical Debt)	Cohort only	0.204***	(0.057)	0.001

Note: This table examines the sensitivity of the main firm-level results to the choice of standard error clustering. Each row reports the Column (1) baseline stacked DiD specification (± 12 month window, firm-by-cohort and event-time-by-cohort fixed effects) with a different clustering structure. The point estimate is mechanically identical across structures; only standard errors change. The main results use firm-level clustering, which accounts for serial correlation within firm and cross-cohort correlation from control firms appearing in multiple cohorts. * $p < 0.1$, ** $p < 0.05$, *** $p < 0.01$.

⁶Wild cluster bootstrap inference, which provides small-cluster-robust p -values, is computationally infeasible in our setting due to the high-dimensional absorbed fixed-effects structure (firm $\times$ cohort and time $\times$ cohort). The clustering sensitivity analysis reported here provides the relevant evidence on robustness to alternative variance estimation approaches.

Dose-Response Heterogeneity. We examine whether treatment effects vary with the complexity of the required remediation. Table 25 stratifies by the severity of the required dependency upgrade (major versus minor version changes, following semantic versioning conventions). Major upgrades—which by definition introduce breaking API changes—show larger headcount disruption (-0.092 , $p = 0.016$) than minor upgrades (-0.075 , $p = 0.111$). For technical debt, both upgrade types yield significant effects of similar magnitude (0.185 vs. 0.217); a pooled interaction model fails to reject the null of equal treatment effects ($F = 0.10$, $p = 0.755$). These dose-response patterns support the interpretation that our treatment effects operate through the remediation mechanism rather than through an information or reputational channel, as the latter would predict effects proportional to exploitability rather than remediation complexity.

Table 25: Dose-Response Heterogeneity: Upgrade Severity

Upgrade Type	Outcome	Coefficient	SE	<i>p</i> -Value
Major	Ln(Headcount)	-0.092^{**}	(0.036)	0.016
Minor	Ln(Headcount)	-0.075	(0.046)	0.111
Major	Ln(Technical Debt)	0.185^{**}	(0.078)	0.024
Minor	Ln(Technical Debt)	0.217^{**}	(0.092)	0.024

Note: This table examines whether treatment effects vary with the complexity of the required remediation. Treated firms are stratified by the severity of the required dependency upgrade following semantic versioning conventions: major upgrades introduce breaking API changes, while minor upgrades are backward-compatible. The control group is the full set of untreated firms; only the treated subsample varies. All specifications use the Column (1) baseline stacked DiD (± 12 month window, firm-by-cohort and event-time-by-cohort fixed effects). Standard errors in parentheses are clustered at the firm level. * $p < 0.1$, ** $p < 0.05$, *** $p < 0.01$.

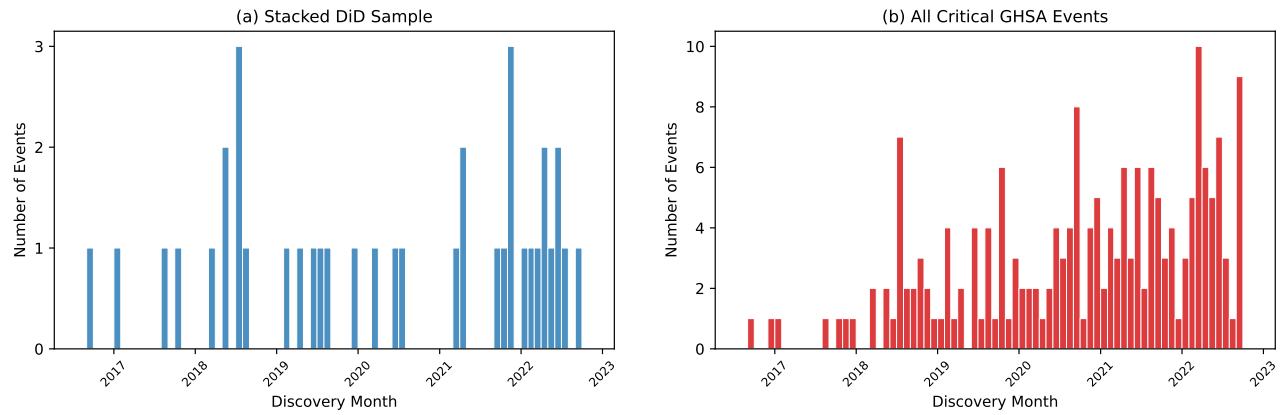


Figure 13: Distribution of Critical Vulnerability Events Over Time. Panel (a) shows the 38 cohort events used in our stacked DiD sample. Panel (b) shows all 189 critical-severity events ($CVSS \geq 9$) recorded in the GitHub Advisory Database over the same period. Both panels share the same time axis (September 2016 to September 2022). The temporal distribution of our DiD cohorts broadly mirrors the rising trend in critical vulnerability discoveries, suggesting that our sample of shocks is representative of the general population of critical disclosures rather than concentrated in an unusual period.

Table 26: Sample Attrition

Restriction	Firms	Firms Dropped
OSSRank universe	648	—
Successful repository download	636	12
Valid SonarQube metrics	488	148
Single repository, headcount > 1	298	190
DiD sample (stacked)	295	3

Note: This table documents sample construction. *OSSRank universe* includes all 648 open-core companies identified via OSSRank rankings. *Successful repository download* includes the 636 firms for which we successfully matched and cloned at least one primary GitHub or GitLab repository; the remaining 12 firms had repositories that were private, deleted, or otherwise inaccessible. *Valid SonarQube metrics* restricts to firms with at least one month of non-zero code metrics (NCLOC > 0), excluding repositories whose programming languages or build configurations are not supported by our static analysis pipeline. *Single repository, headcount > 1* retains firms that commercialize exactly one open-source repository, so that codebase metrics unambiguously capture the firm’s core product, and that have peak headcount exceeding one employee, excluding individual-developer side projects. *DiD sample (stacked)* applies the treatment definition: treatment is the firm’s first critical-severity dependency vulnerability requiring a major or minor version upgrade under semantic versioning; control firms must have no such vulnerability during the relevant ± 12 month event window. The final sample includes 295 firms appearing in at least one of 38 vulnerability cohorts satisfying these criteria.